

BP Internet Banking

1 Executive Summary

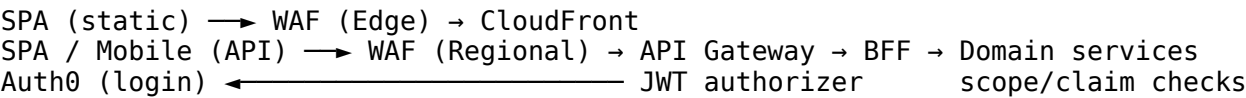
Scope: Ecuador retail digital banking — movement history, own-account and interbank transfers, mobile onboarding with facial recognition, regulatory notifications.

Stack: React SPA + React Native on **AWS** (**us-east-1** primary, **us-west-2** DR). Identity via **Auth0** (Authorization Code + PKCE).

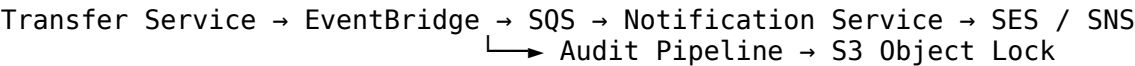
1.1 C4 at a glance

Level	View	What it shows
Context	BP-Context	Customer, BP platform, Auth0, Core, Detail, SWIFT, SES, SNS, Rekognition
Containers	BP-Containers	Channels → API Gateway → BFF → 6 microservices → Redis, EventBridge, SQS, S3 audit, DynamoDB, CloudWatch, X-Ray
Components	BP-Components-*	BFF, Profile, Movements, Transfer, Onboarding, Notification, Audit, Risk (8 views)

1.2 Request path



Transfer side-effects (async):



1.3 Key decisions (ADRs)

Topic	ADR	Decision
Auth	ADR-001	Auth0, Authorization Code + PKCE
JWT split	ADR-003	Gateway validates token; BFF checks scopes/ acr
Integration	ADR-003 , ADR-004	API Gateway + BFF + ACL adapters
Transfers	ADR-011	Saga orchestration, idempotency
Notifications & audit	ADR-012	EventBridge fan-out via SQS
Cache	ADR-010	Cache-Aside (Redis) for frequent clients
Onboarding	ADR-008	Rekognition liveness + Auth0 provisioning
Observability	ADR-015	CloudWatch + X-Ray (5% sampled)
HA / DR	ADR-007	Multi-AZ + warm standby us-west-2

Full index: [Decisions tab](#) (15 ADRs).

2 Architecture Overview

2.1 Purpose

This document describes the solution architecture for **BP Internet Banking**, a digital banking platform that allows retail customers in Ecuador to:

- View transaction movement history
- Execute transfers and payments between own accounts and interbank accounts (via SWIFT)

The architecture follows the **C4 model** (Context → Containers → Components), is deployed on **AWS**, and integrates with legacy Core banking systems, Auth0, and external payment/notification providers.

2.2 System Context Summary

The BP Internet Banking system sits between retail customers and several external systems:

Actor / System	Role
Retail Banking Customer	Uses web (React SPA) or mobile (React Native)
Auth0	OAuth 2.0 / OIDC identity provider, MFA, Universal Login
Core Banking Platform	Legacy REST — client basics, products, movements, account operations
Client Detail System	Legacy REST — extended client information on demand
SWIFT Payment Hub	Correspondent bank hub for interbank transfers
Amazon SES	Email notifications (regulatory movement alerts)
Amazon SNS / Pinpoint	SMS notifications
Amazon Rekognition	Facial liveness and enrollment for mobile onboarding

See Structurizr view: ****BP-Context****.

2.3 Container Architecture Summary

2.3.1 Client Channels

Container	Technology	Purpose
React SPA	React 18, TypeScript	Web banking channel
React Native Mobile App	React Native + Kotlin native modules	iOS/Android with secure biometrics

Both channels authenticate via **Auth0** (Authorization Code + PKCE). **Static assets:** SPA → WAF (Edge) → CloudFront. **API calls:** SPA and mobile → WAF (Regional) → API Gateway → BFF.

2.3.2 Integration Layer

Container	Purpose
AWS WAF (Edge)	Web ACL (Web Access Control List) on CloudFront — OWASP rules, bot control, geo restrictions
Amazon CloudFront	Global CDN and TLS edge for static asset delivery
AWS WAF (Regional)	Web ACL on API Gateway — rate limiting, IP reputation, OWASP rules
Amazon API Gateway	API entry point, throttling, JWT validation, routing
Banking BFF	Aggregates domain services, shapes API for clients, MFA orchestration

Pattern: API Gateway + BFF + Anti-Corruption Layer (see [ADR-003](#), [ADR-004](#)).

2.3.3 Domain Microservices

Service	Responsibility	Stack
Customer Profile Service	Basic + detail client data, cache-aware	Kotlin / Spring Boot
Movements Service	Paginated movement history	Kotlin / Spring Boot
Transfer Service	Own-account and SWIFT interbank transfers	Kotlin / Spring Boot
Onboarding & Identity Service	Facial enrollment, device binding, Auth0 provisioning	Kotlin / Spring Boot
Risk & MFA Gate Service	Risk scoring, step-up MFA triggers	Kotlin / Spring Boot
Notification Orchestrator	Async email + SMS for movements	Kotlin / Spring Boot

Justified additions beyond the 3 required services:

1. **Onboarding & Identity Service** — Required by assignment for facial onboarding flow; isolates **PII (Personally Identifiable Information)** and biometric processing from core banking paths.
2. **Risk & MFA Gate Service** — Separates fraud logic from transfer execution (Single Responsibility); enables independent rule updates without redeploying Transfer Service.
3. **Notification Orchestrator** — Decouples notification delivery from transfer path (async, retries, regulatory dual-channel compliance).

2.3.4 Data & Messaging

Container	Pattern / Purpose
ElastiCache Redis	Cache-Aside for frequent client profiles and recent movements
DynamoDB	Idempotency keys, device registry, onboarding state, audit index
EventBridge	Domain event bus (audit, notifications)
SQS + DLQ	Reliable async processing with fault isolation
S3 Object Lock	Immutable audit store, 2-year compliance retention
Audit Event Pipeline	Lambda consumers normalizing and persisting audit records
Secrets Manager	Centralized credential management

See Structurizr view: ****BP-Containers****.

2.4 Component-Level Highlights

2.4.1 Banking BFF (BP-Components-BFF)

- **Auth Middleware** — Scope and claim authorization (JWT validated at API Gateway)
- **MFA Orchestrator** — Coordinates with Risk Gate before high-value transfers
- **Response Aggregator** — Combines Core + Detail data in one round-trip
- **Cache Client** — Cache-Aside for hot reads

2.4.2 Customer Profile Service (BP-Components-Profile)

- **Core Banking Adapter** — Anti-corruption layer over legacy Core REST
- **Detail System Adapter** — On-demand extended client data from Detail system
- **Core / Detail Circuit Breakers** — Resilience4j with cached fallback for reads

2.4.3 Movements Service (BP-Components-Movements)

- **Core Banking Adapter** — Anti-corruption layer over Core movements API
- **Core Circuit Breaker** — Resilience4j; serves cached movements when Core degrades

2.4.4 Transfer Service (BP-Components-Transfer)

- **Saga Orchestrator** — Distributed transaction across Core debit and SWIFT hub
- **Idempotency Guard** — DynamoDB-backed duplicate prevention
- **Core Banking Adapter** — Anti-corruption layer over legacy REST
- **SWIFT Hub Adapter** — mTLS integration with correspondent bank

2.4.5 Onboarding & Identity (BP-Components-Onboarding)

- **Facial Enrollment Service** — Rekognition liveness + template storage
- **Device Binding Service** — Trusted device registry for biometric login
- **Auth0 Provisioner** — Links face/password/biometric methods post-enrollment

2.4.6 Audit Pipeline (BP-Components-Audit)

- **Immutable Writer** — S3 Object Lock (Compliance mode), append-only
- **Audit Normalizer** — Canonical schema: who, what, when, correlationId, IP, device

2.4.7 Risk & MFA Gate (BP-Components-Risk)

- **Rule Engine** — Amount thresholds, velocity, new payee, device trust
- **MFA Trigger** — Auth0 step-up for high-risk operations

2.5 Key Data Flows

2.5.1 View Movements (read path)

Customer → SPA/Mobile → WAF (Regional) → API Gateway → BFF
 → [Cache hit?] Redis → return cached
 → [Cache miss] Movements Service → Core REST → cache write → return

2.5.2 Load SPA (static path)

Customer → SPA → WAF (Edge) → CloudFront → static assets

2.5.3 Execute Transfer (write path)

Customer → BFF → Risk Gate → [MFA required?] Auth0 step-up
 → Transfer Service → Idempotency check → Saga:
 1. Core Adapter: debit source account
 2. [if interbank] SWIFT Adapter: submit to payment hub
 3. Core Adapter: credit destination (if applicable)
 → EventBridge: transfer.completed
 → Audit Pipeline → S3 Object Lock
 → Notification Orchestrator → SES + SNS

2.5.4 Mobile Onboarding

Customer → Mobile App → BFF → Onboarding Service
 → Rekognition: liveness + face enrollment
 → Device Binding → DynamoDB
 → Auth0 Provisioner: create user, enable login methods

2.6 Architecture Patterns Applied

Pattern	Where	Rationale
BFF	Banking BFF	Optimizes API for each channel without polluting domain services
Anti-Corruption Layer	Core/SWIFT adapters	Isolates domain from legacy API shapes
Cache-Aside	Redis + Profile/Movements	Low latency for frequent clients
Saga	Transfer Service	Consistent state across Core + SWIFT without 2PC
Event-Driven	EventBridge	Decouples audit and notifications from transfer latency path
Circuit Breaker	All legacy adapters	Fault tolerance when Core/Detail degrade
Idempotency	Transfer Service	Safe retries at millions of tx/day scale
CQRS (light)	Separate read (Movements) and write (Transfer) services	Independent scaling of read vs write paths
Kotlin domain tier	All six domain microservices	One JVM language; Spring Boot; null safety; shared libraries with mobile Kotlin modules

2.7 AWS Deployment Topology

Region	Role	Services
us-east-1	Primary (multi-AZ)	All active workloads: ECS Fargate, API Gateway, Redis, DynamoDB global tables, EventBridge
us-west-2	DR (warm standby)	Pre-provisioned ECS task definitions, replicated S3 audit buckets, DynamoDB global table replicas, Route 53 health-check failover

Target SLA: 99.95% (~4.38 hours downtime/year).

2.8 Structurizr Diagram Reference

All diagrams are defined in [structurizr/workspace.dsl](../structurizr/workspace.dsl).

View	Description
BP-Context	Non-technical system context
BP-Containers	Technical container view with AWS building blocks
BP-Components-BFF	BFF internal components
BP-Components-Profile	Profile Service Core/Detail adapters
BP-Components-Movements	Movements Service legacy integration
BP-Components-Transfer	Transfer Service with SWIFT integration
BP-Components-Onboarding	Facial enrollment and device binding
BP-Components-Notification	Email + SMS orchestration
BP-Components-Audit	Immutable audit pipeline
BP-Components-Risk	Risk scoring and MFA gate

2.9 Related Documents

- [Authentication Architecture](#)
- [Mobile Framework Comparison](#)
- [Non-Functional Requirements](#)
- [Regulatory Context](#)
- [Cost Estimate](#)
- [ADR Index](#)

3 Authentication & Authorization Architecture

3.1 Overview

BP Internet Banking uses **Auth0** as the OAuth 2.0 / OpenID Connect identity provider. Both the React SPA and React Native mobile app authenticate through Auth0 Universal Login, with custom security logic implemented via **Auth0 Actions** and the **Risk & MFA Gate Service**.

3.2 Recommended OAuth 2.0 Flow

3.2.1 Primary flow: Authorization Code + PKCE

Client	Flow	Justification
React SPA	Authorization Code + PKCE	Public client cannot store secrets; PKCE prevents authorization code interception (RFC 7636, OAuth 2.0 Security BCP)
React Native Mobile	Authorization Code + PKCE	Same rationale; Auth0 SDK handles code verifier/challenge

Why not Implicit or Client Credentials?

- **Implicit flow** is deprecated (OAuth 2.0 Security BCP, RFC 9700) — tokens exposed in browser history.
- **Client Credentials** is for machine-to-machine, not end-user banking.

Why not Resource Owner Password Credentials?

- Credentials pass through the client app — violates least privilege and PCI/security best practices.
- No MFA integration at credential exchange point.

3.2.2 Token strategy

Token	Lifetime	Storage
Access Token (JWT)	5–15 minutes	Memory only (SPA); secure storage (mobile)
Refresh Token	Rotating, 24h	HttpOnly cookie (SPA via BFF) or Keychain/Keystore (mobile)

Token	Lifetime	Storage
ID Token	Same as access	Used for client-side identity display only, never for API auth

Refresh Token Rotation (Auth0 built-in): Each refresh issues a new refresh token and invalidates the previous one, limiting replay attack window (OAuth 2.0 Security BCP).

3.3 Auth0 Configuration

3.3.1 Applications

Auth0 Application	Type	Grant
BP Web SPA	Single Page Application	Authorization Code + PKCE
BP Mobile App	Native	Authorization Code + PKCE

3.3.2 Auth0 Actions (custom security logic)

Action	Trigger	Purpose
Enforce Device Binding	Post-Login	Validates device fingerprint against DynamoDB registry
Risk Score Check	Post-Login	Calls Risk & MFA Gate; denies or challenges suspicious logins
Enforce MFA	Post-Login / Step-Up	Requires MFA when risk score exceeds threshold
Audit Login Event	Post-Login	Publishes login event to EventBridge for immutable audit
Token Customization	Post-Login	Adds bp_customer_id, device_id, auth_method claims

3.3.3 MFA Methods (Auth0 Guardian)

- **OTP** (TOTP via authenticator app) — step-up for high-value transfers
- **SMS OTP** — fallback (lower assurance, regulated availability)
- **Push notification** — optional future enhancement

3.4 Login Methods After Onboarding

After mobile onboarding, customers may authenticate using:

Method	Technology	When
Username + Password	Auth0 database connection	Any channel
Fingerprint / Face ID	Device biometric APIs via Kotlin native module (Android) / LocalAuthentication (iOS)	Mobile — unlocks stored refresh token
Facial recognition (cloud)	Amazon Rekognition compare against enrolled template	Mobile — optional explicit login choice

3.4.1 Biometric login flow (mobile)

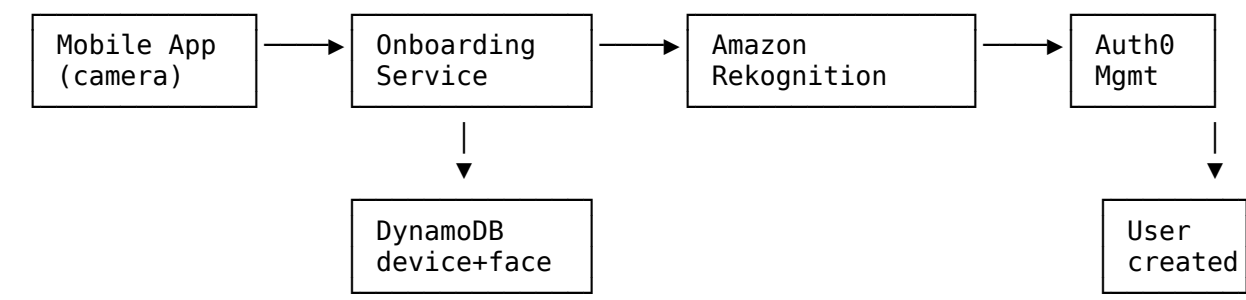
1. User selects biometric login
2. Kotlin native module invokes Android BiometricPrompt / iOS LocalAuthentication
3. On success: retrieve refresh token from Android Keystore / iOS Keychain
4. Exchange refresh token with Auth0 for new access token (PKCE session)
5. Auth0 Action validates device binding
6. Access token sent to API Gateway → BFF

Industry tools:

Tool	Purpose
react-native-keychain	Secure token storage (Keychain / Keystore)
react-native-biometrics	Unified biometric API
Kotlin BiometricPrompt	Hardware-backed biometric on Android (native module)
Android Keystore / iOS Secure Enclave	Hardware security module for key material
Auth0 Guardian	MFA and step-up challenges

Tool	Purpose
Amazon Rekognition Face Liveness	Presentation attack detection at onboarding

3.5 Onboarding Flow (Mobile)



3.5.1 Steps

- 1. Customer provides basic identity data (national ID, phone, email)
- 2. **Liveness check** — Rekognition Face Liveness prevents photo/video spoofing
- 3. **Face enrollment** — template stored encrypted in DynamoDB (not raw image in app)
- 4. **Device binding** — device fingerprint registered
- 5. **Auth0 provisioning** — user created via Management API; password set; biometric methods enabled
- 6. **Audit event** — onboarding.completed written to immutable store

3.6 Step-Up MFA for High-Risk Operations

Triggered by **Risk & MFA Gate Service** before transfer execution:

Condition	Action
Transfer amount > configurable threshold	Auth0 step-up MFA (OTP)
New payee (first transfer to account)	Step-up MFA
Login from unrecognized device	Deny or MFA at login (Auth0 Action)
Velocity exceeded (N transfers in M minutes)	Block + alert

3.6.1 Step-up flow

BFF → Risk Gate: `evaluate(transferContext)`
Risk Gate → `score >= threshold?`
YES → BFF returns `403 + mfa_required` challenge URL
Client → Auth0 step-up authentication (`acr_values`)
Client retries transfer with elevated token (`acr` claim verified by BFF)
NO → proceed to Transfer Service

Theoretical basis:

- 1. **NIST SP 800-63B** — Authenticator Assurance Levels; step-up satisfies AAL2 for high-risk transactions.
- 2. **OWASP ASVS V2** — Session management requires re-authentication for sensitive operations.

3.7 API Security

Layer	Mechanism
Transport	TLS 1.3 everywhere; mTLS for SWIFT hub
API Gateway	JWT authorizer (Auth0 JWKS), rate limiting, WAF
BFF	Scope validation (<code>read:movements,write:transfers</code>), correlation IDs
Service-to-service	IAM roles (ECS task roles), no static credentials
Secrets	AWS Secrets Manager with automatic rotation

3.8 Sequence: Complete Login + Transfer

6/19/26, 9:00 PM

Structurizr - BP Internet Banking

Customer → Auth0 Universal Login (PKCE)
Auth0 → Actions: device check, risk check, MFA if needed
Auth0 → Customer: access_token + refresh_token
Customer → API Gateway (Bearer token)
API Gateway → JWT validation
BFF → Movements Service (read, cached)
Customer → BFF: POST /transfers
BFF → Risk Gate: evaluate
Risk Gate → MFA required → Customer completes step-up
BFF → Transfer Service: execute saga
Transfer Service → EventBridge → SQS (notification queue) → Notification Service → SES / SNS
 ↳ Audit Pipeline → S3

3.9 Related ADRs

- [ADR-001: Auth0 and Authorization Code + PKCE](#)
- [ADR-005: MFA Step-Up for High-Risk Operations](#)
- [ADR-008: Mobile Onboarding with Facial Recognition](#)

4 Mobile Framework Comparison

The assignment requires evaluating **two multiplatform mobile framework options** and justifying the final choice.

4.1 Candidates

Criteria	React Native	Flutter
Vendor	Meta (open source)	Google (open source)
Language	JavaScript / TypeScript (+ native modules)	Dart
UI approach	Native components via bridge	Custom rendering engine (Skia)
Code sharing	~85–95% iOS/Android	~95%+ iOS/Android
Ecosystem	npm, large React community	pub.dev, growing community
Banking adoption	Nubank, Coinbase, Bloomberg	Alibaba, BMW, ING

4.2 Evaluation Matrix

Criterion	Weight	React Native	Flutter	Notes
Security / biometrics	High	★★★★☆	★★★★☆	RN allows Kotlin/Swift native modules for Keystore, BiometricPrompt — critical for banking
Team alignment (React SPA)	High	★★★★★	★★☆☆☆	Shared TypeScript skills with React SPA
Performance	Medium	★★★★☆	★★★★★	Flutter slightly faster UI; RN New Architecture closes gap
Native API access	High	★★★★★	★★★★☆	RN native modules are first-class; Flutter uses platform channels
Maturity in fintech	High	★★★★★	★★★★☆	RN has more proven banking deployments in LATAM
OTA updates	Medium	★★★★☆ (CodePush)	★★☆☆☆	RN supports hotfix without app store review (with limits)
Facial recognition SDK	High	★★★★★	★★★★☆	Both integrate AWS Rekognition; RN Kotlin module for camera control
Developer availability	Medium	★★★★★	★★★★☆	Larger React/TS talent pool in Ecuador/LATAM
UI consistency	Low	★★★☆☆	★★★★★	Flutter renders identically; RN uses platform-native widgets

4.3 Security-First Consideration

Banking apps require **hardware-backed secure storage** and **biometric APIs** that are best accessed through thin native layers:

4.3.1 React Native approach (chosen)

React Native (TypeScript)
Kotlin Native Module (Android) Swift Native Module (iOS)

UI, navigation, business logic

BiometricPrompt, Keystore, root detection
LocalAuthentication, Keychain

Advantages:

- 1. **Defense in depth** — Sensitive crypto operations never touch the JS bridge; Kotlin handles Keystore directly.
- 2. **Platform security updates** — Biometric APIs evolve per OS; native modules absorb changes without rewriting the UI layer.
- 3. **Auditability** — Security-critical code isolated in small, reviewable native modules.

4.3.2 Flutter approach (evaluated, not chosen)

- Platform channels achieve similar native access but add serialization overhead and complexity for security-critical paths.
- Dart codebase diverges from React SPA — no skill reuse.
- Excellent for greenfield apps without an existing React web team.

4.4 Decision

Selected: React Native with Kotlin native security modules

4.4.1 Justification 1: Shared competence (Conway's Law)

The web channel uses React. Using React Native allows the same team to share components, API client libraries, TypeScript types, and testing practices. This reduces coordination overhead and defect rate (Brooks's Law — adding communication paths increases cost).

4.4.2 Justification 2: Native security module pattern

React Native's bridge architecture explicitly supports **native modules** for security-sensitive operations. Kotlin's **BiometricPrompt** and Android Keystore provide hardware-backed key storage required by banking regulators. Flutter's platform channels are capable but introduce an additional abstraction layer for the most security-critical code paths.

4.4.3 Justification 3: Proven fintech track record

React Native is used by Nubank (largest digital bank in LATAM), demonstrating viability at banking scale with regulatory compliance in Brazilian context (analogous to Ecuador).

4.4.4 Alternatives considered and rejected

Option	Rejection reason
Flutter	No skill overlap with React SPA; security modules less ergonomic for Kotlin-first Android strategy
Native iOS (Swift) + Native Android (Kotlin)	Highest security but 2x development cost; violates multiplatform requirement
Ionic / Capacitor	WebView-based; inferior biometric integration and performance for banking UX
PWA only	Cannot meet facial enrollment, biometric login, or app store distribution requirements

4.5 Recommended Libraries

Library	Purpose
react-native-keychain	Secure token storage
react-native-biometrics	Fingerprint / Face ID
Custom Kotlin module	BiometricPrompt, root/jailbreak detection, certificate pinning
react-native-auth0	Official Auth0 SDK with PKCE
react-native-vision-camera	Camera for facial enrollment
react-native-ssl-pinning	TLS certificate pinning

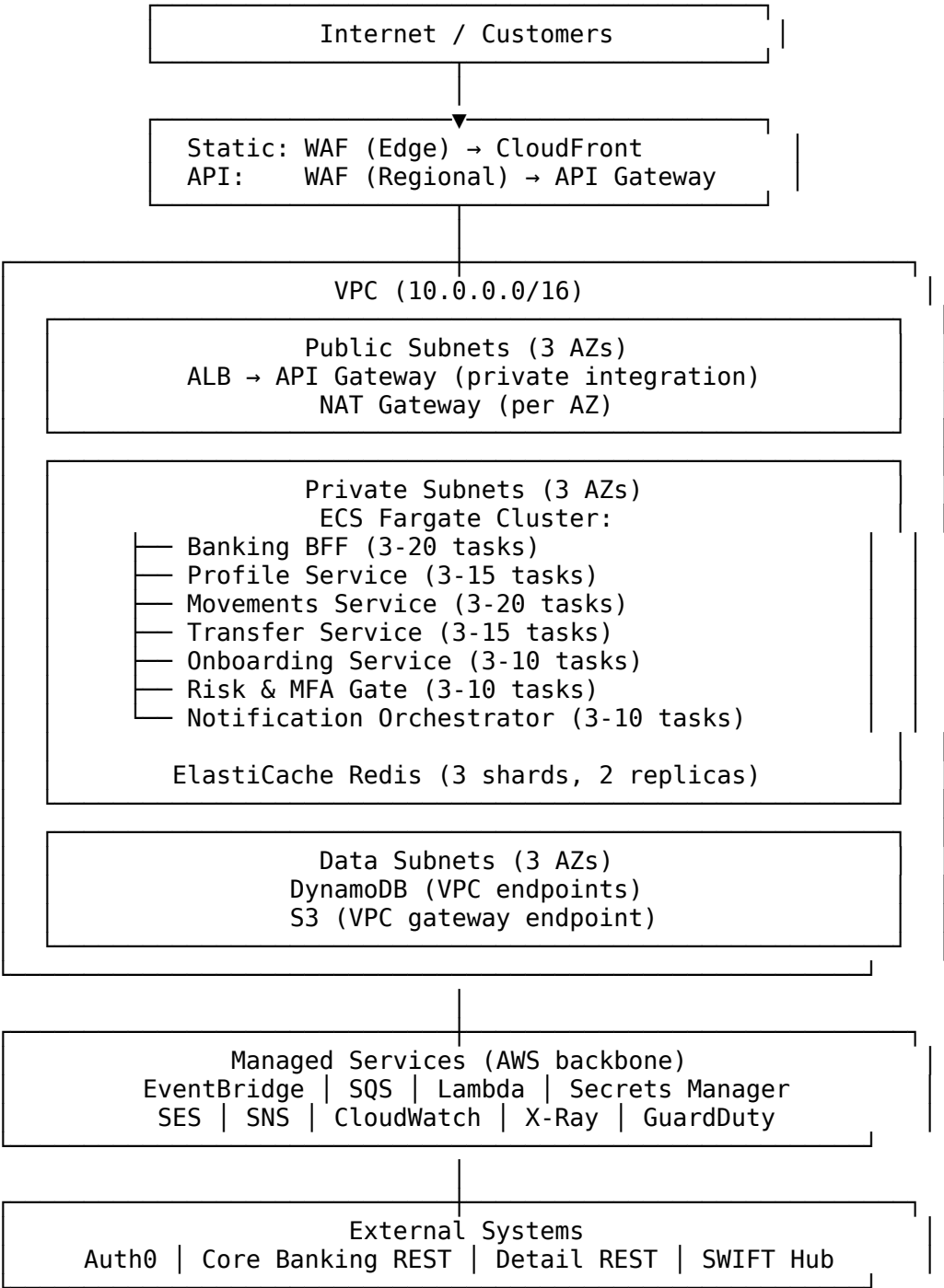
4.6 Related ADR

- [ADR-006: React Native for Mobile](#)

5 Deployment Architecture

Complements the C4 Container diagram with infrastructure topology for the primary (us-east-1) and DR (us-west-2) regions.

5.1 Primary Region — us-east-1



5.2 DR Region — us-west-2 (Warm Standby)

Component	DR State	Failover Action
ECS Fargate	1 task per service (minimum)	Auto-scale to production capacity

Component	DR State	Failover Action
API Gateway	Deployed, Route 53 weighted 0% DNS failover to 100%	
DynamoDB	Global table replica	Automatic read/write promotion
S3 Audit	CRR replica	Read-only until promotion
ElastiCache	Replica cluster (smaller)	Promote replica to primary
EventBridge	Secondary event bus	Activate routing rules
CloudFront	Same distribution (global)	No change needed

5.3 Network Security

Control	Implementation
VPC (Virtual Private Cloud) isolation	All ECS (Elastic Container Service) tasks in private subnets, no public IPs
Security groups	Least privilege: BFF → services only, services → Redis/DynamoDB only
NACLs (Network Access Control Lists)	Default deny inbound from internet to private subnets
VPC endpoints	S3, DynamoDB, SQS, Secrets Manager, CloudWatch (no NAT for AWS traffic)
TLS (Transport Layer Security)	TLS 1.3 termination at CloudFront and ALB (Application Load Balancer)
mTLS (mutual Transport Layer Security)	SWIFT hub communication

5.4 Auto-Scaling Policies

Service	Scale Trigger	Min	Max
Banking BFF	ALB request count > 1000/min/task	3	20
Movements Service	CPU > 70% or cache miss rate > 30%	3	20
Transfer Service	SQS depth > 100 or CPU > 70%	3	15
Notification Orchestrator	SQS queue depth > 500	3	15
All others	CPU > 70%	3	10

5.5 CI/CD Pipeline

GitHub → CodePipeline → CodeBuild (test + build Docker image)
 → ECR (container registry)
 → CodeDeploy (ECS blue/green)
 → CloudWatch (deployment health check)
 → Auto-rollback on error rate spike

5.6 Route 53 Failover

Record	Primary	DR
api.bp-banking.com	us-east-1 API Gateway (health check)	us-west-2 API Gateway (failover)
Health check interval	30 seconds	—
Failover threshold	3 consecutive failures	—

6 Non-Functional Requirements

6.1 Service Level Objectives

Metric	Target	Measurement
Availability	99.95%	Uptime for customer-facing APIs (excludes planned maintenance windows)
API latency (p99)	< 500 ms reads, < 2 s writes	CloudWatch + X-Ray
RTO (Recovery Time Objective)	< 30 minutes	DR failover to us-west-2
RPO (Recovery Point Objective)	< 5 minutes	DynamoDB global tables, S3 cross-region replication

Metric	Target	Measurement
Throughput	Tens of millions tx/day peak	Auto-scaling validated via load tests

6.2 High Availability (HA)

6.2.1 Multi-AZ within us-east-1 (primary)

Component	HA Strategy
ECS Fargate	Minimum 3 tasks per service across 3 AZs; ALB health checks
API Gateway	Managed regional service (inherently multi-AZ)
ElastiCache Redis	Cluster mode enabled, 3 shards × 2 replicas across AZs
DynamoDB	On-demand capacity, multi-AZ by default
EventBridge	Managed, multi-AZ
S3 Audit	99.999999999% durability, multi-AZ
CloudFront	Global edge network

6.2.2 Justifications

1. Bulkhead pattern — Each microservice scales independently; failure in Notification Service does not block Transfer Service.
2. N+1 redundancy — Minimum 3 instances per service ensures survival of single-AZ failure (AWS Well-Architected Reliability pillar).

6.3 Fault Tolerance

6.3.1 Patterns implemented

Pattern	Implementation	Failure handled
Circuit Breaker	Resilience4j in Spring Boot adapters	Core/Detail system timeout or 5xx
Retry with backoff	AWS SDK defaults + SQS visibility timeout	Transient network failures
Dead Letter Queue	SQS DLQ for notifications and audit	Poison messages isolated
Idempotency	DynamoDB idempotency keys in Transfer Service	Duplicate transfer submissions
Saga with compensation	Transfer Service orchestrator	Partial failure (debit succeeded, SWIFT failed → reversal)
Graceful degradation	BFF returns cached profile if Detail system down	Detail system unavailable
Timeout budgets	3s Core reads, 10s SWIFT submission	Cascading latency prevention

6.3.2 Legacy integration resilience

Core Banking REST call

→ Circuit Breaker (closed → open after 5 failures in 30s)

→ Half-open: probe every 60s

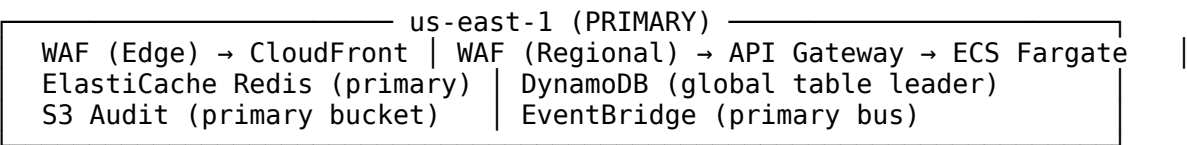
→ Fallback: return cached data (reads) or fail fast with 503 (writes)

Theoretical basis:

1. Michael Nygard — Release It! — Circuit breaker prevents cascading failure in distributed systems.
2. CAP theorem — During partition, prefer **availability** for reads (cached) and **consistency** for writes (fail rather than duplicate money).

6.4 Disaster Recovery (DR)

6.4.1 Topology



| continuous replication

us-west-2 (DR)

ECS Fargate (warm standby: min 1 task per service)	
ElastiCache Redis (replica)	DynamoDB (global table replica)
S3 Audit (CRR replica)	EventBridge (secondary bus)

6.4.2 Failover mechanism

Step	Action
1	Route 53 health check detects us-east-1 API Gateway failure
2	DNS failover shifts traffic to us-west-2 API Gateway endpoint
3	ECS scales warm standby to production capacity (pre-defined auto-scaling)
4	DynamoDB global tables serve reads/writes from us-west-2
5	S3 CRR provides audit data continuity
6	Runbook executed: notify ops, validate SWIFT hub connectivity from DR region

6.4.3 DR testing

- **Quarterly** failover drill to us-west-2
- **Annual** full DR simulation including SWIFT test transactions

Theoretical basis:

1. **ISO 22301** — Business continuity requires tested recovery procedures.
2. **AWS Well-Architected** — **Reliability** — DR strategy must match RTO/RPO; warm standby balances cost and recovery speed.

6.5 Security

6.5.1 Defense in depth

Layer	Controls
Edge	CloudFront, AWS WAF (OWASP Top 10 rules), geo-blocking, rate limiting
API	API Gateway JWT authorizer, throttling (10K req/s burst), request validation
Application	Input validation, OWASP ASVS L2, parameterized queries, CSRF protection (SPA)
Transport	TLS 1.3, certificate pinning (mobile), mTLS (SWIFT hub)
Identity	Auth0 MFA, step-up, device binding, refresh token rotation
Data at rest	AES-256 (S3, DynamoDB, Redis encryption in transit and at rest)
Secrets	AWS Secrets Manager, no credentials in code or env files
Network	VPC private subnets for ECS, no public IPs on services, VPC endpoints for AWS APIs
Audit	Immutable S3 Object Lock, all client actions logged

6.5.2 Threat model highlights

Threat	Mitigation
Credential stuffing	WAF rate limiting + Auth0 breached password detection
MITM	TLS 1.3 + certificate pinning
Session hijacking	Short-lived tokens, refresh rotation, device binding
Face spoofing	Rekognition Face Liveness at enrollment
Duplicate transfers	Idempotency keys
Insider threat	IAM least privilege, CloudTrail, audit pipeline

6.6 Monitoring & Observability

6.6.1 Three pillars

Pillar	Tool	Purpose
Metrics	Amazon CloudWatch	SLIs, SLOs, custom business metrics (transfers/min, cache hit rate)
Logs	CloudWatch Logs + structured JSON	Centralized, searchable application logs
Traces	AWS X-Ray	Distributed tracing across BFF → services → legacy adapters

6.6.2 Key dashboards

Dashboard	Metrics
Customer Experience	API p50/p99 latency, error rate, availability
Transfer Health	Transfer success rate, saga compensation rate, SWIFT hub latency
Security	Failed logins, MFA challenges, WAF blocks, GuardDuty findings
Infrastructure	ECS CPU/memory, Redis hit rate, DynamoDB throttles
Business	Daily transaction volume, notification delivery rate

6.6.3 Alerting

Severity	Example	Response
P1 — Critical	API availability < 99.9% over 5 min	PagerDuty → on-call SRE
P2 — High	Transfer error rate > 1%	Slack + auto-scale investigation
P3 — Medium	Cache hit rate < 80%	Ticket to platform team
P4 — Low	Certificate expiry < 30 days	Automated renewal (ACM)

6.6.4 Security monitoring

Service	Purpose
AWS GuardDuty	Threat detection (anomalous API calls, compromised instances)
AWS Security Hub	Compliance posture aggregation
AWS Config	Configuration drift detection
CloudTrail	API audit trail for all AWS management actions

6.7 Operational Excellence & Auto-Healing

6.7.1 Auto-healing mechanisms

Component	Auto-healing
ECS Fargate	Unhealthy tasks replaced automatically by ALB health checks
Auto Scaling	CPU > 70% or custom metric → scale out; scale in after cooldown
Lambda (Audit)	Failed invocations → automatic retry (2x) → DLQ
SQS consumers	Visibility timeout → redelivery; DLQ after max receives
ElastiCache	Automatic failover to replica on primary failure
DynamoDB	Automatic partition management
Certificate Manager	Auto-renewal of TLS certificates

6.7.2 CI/CD and deployment

Practice	Implementation
Blue/Green deployment	ECS CodeDeploy blue/green for zero-downtime releases
Canary releases	10% traffic to new version, auto-rollback on error spike
Infrastructure as Code	AWS CDK / Terraform for reproducible environments
Automated testing	Unit → integration → contract tests → load tests in pipeline

6.7.3 Runbooks

- DR failover procedure

- Core banking outage (circuit breaker + customer communication)
- SWIFT hub timeout (saga compensation)
- Auth0 outage (cached JWKS, degraded login)

Theoretical basis:

1. **Google SRE — Error budgets** — 99.95% SLA allows ~4.38h downtime/year; error budget drives release velocity decisions.
2. **Chaos Engineering (Principles of Chaos)** — Quarterly fault injection validates auto-healing (e.g., kill ECS tasks, simulate AZ failure).

6.8 Performance & Low Latency

Technique	Impact
CloudFront edge caching (static assets)	Reduces SPA load time for Ecuador users
Redis Cache-Aside (profile, movements)	Sub-10ms reads for frequent clients
BFF aggregation	Single round-trip instead of 3+ client calls
DynamoDB on-demand	No cold-start capacity planning at peak
Connection pooling	Reuse HTTP connections to legacy REST APIs
Async notifications	Transfer response not blocked by email/SMS delivery

6.9 Related ADRs

- [ADR-007: Multi-Region DR Strategy](#)
- [ADR-009: Immutable Audit with S3 Object Lock](#)
- [ADR-010: Cache-Aside with ElastiCache Redis](#)

7 Regulatory & Security Standards

7.1 Ecuador Regulatory Framework

7.1.1 Ley Orgánica de Protección de Datos Personales (LOPDP)

Law: Ley Orgánica de Protección de Datos Personales (published May 2021, effective May 2023).

Requirement	Architecture Response
Lawful basis for processing	Customer consent captured during onboarding; purpose-limited data collection
Data minimization	BFF aggregates only required fields; Detail system queried on demand
Right of access / rectification	Profile Service exposes customer data; audit trail supports access requests
Right to erasure	Soft-delete with audit retention exception (financial records exempt)
Data security	Encryption at rest and in transit, access controls, immutable audit
Data breach notification	EventBridge security events → automated incident response runbook
Cross-border transfer	Customer data in AWS us-east-1/us-west-2 with SCCs and encryption; documented in privacy policy
Data Protection Officer	Organizational requirement; architecture supports audit queries for DPO

7.1.2 Superintendencia de Bancos del Ecuador (SB)

Regulation Area	Requirement	Architecture Response
Cybersecurity	SB cybersecurity guidelines for financial institutions	WAF, GuardDuty, Security Hub, penetration testing
Electronic banking	Secure channels for digital banking operations	TLS 1.3, MFA, device binding, immutable audit
Customer notification	Notify customers of account movements	Dual-channel: SES (email) + SNS (SMS) via Notification Orchestrator
Transaction records	Retain transaction evidence	S3 Object Lock audit store, 2-year minimum retention
Business continuity	DR capability for critical systems	Multi-region warm standby, quarterly DR drills
Third-party risk	Manage vendor risk (cloud, identity)	Auth0 enterprise agreement, AWS shared responsibility model

7.1.3 Banco Central del Ecuador (BCE)

Area	Relevance
Payment system rules	Interbank transfers via SWIFT must comply with BCE payment system regulations
Anti-money laundering (AML)	Risk Gate velocity checks; large transfer reporting hooks
Foreign exchange	SWIFT transfers subject to BCE reporting if applicable

7.2 International Security Standards

7.2.1 PCI-DSS (Payment Card Industry)

While BP Internet Banking primarily handles account transfers (not card processing), PCI-DSS principles apply:

Principle	Application
Network segmentation	VPC private subnets, security groups
Encryption	TLS 1.3, AES-256 at rest
Access control	IAM least privilege, MFA for admin access
Logging & monitoring	Immutable audit, CloudTrail, GuardDuty
Vulnerability management	Automated dependency scanning in CI/CD

7.2.2 ISO/IEC 27001 — Information Security Management

Domain	Architecture Alignment
A.9 Access control	Auth0 RBAC, IAM policies, JWT scopes
A.10 Cryptography	TLS, encryption at rest, Keystore/Keychain
A.12 Operations security	Monitoring, logging, change management (CI/CD)
A.13 Communications security	TLS, mTLS, WAF
A.14 System development	Secure SDLC, code review, SAST/DAST
A.16 Incident management	GuardDuty → automated response, runbooks
A.17 Business continuity	Multi-region DR
A.18 Compliance	Audit pipeline, regulatory mapping (this document)

7.2.3 OWASP

Standard	Application
OWASP Top 10 (2021)	WAF rules, input validation, parameterized queries, CSRF tokens
OWASP ASVS Level 2	Authentication, session management, access control for banking
OWASP MASVS	Mobile app security verification (certificate pinning, root detection)

7.2.4 NIST Cybersecurity Framework

Function	Implementation
Identify	Asset inventory (AWS Config), risk assessment (Risk Gate)
Protect	WAF, encryption, MFA, IAM
Detect	GuardDuty, CloudWatch alarms, audit anomaly detection
Respond	Incident runbooks, automated isolation (Security Hub)
Recover	DR failover, saga compensation, backup restoration

7.2.5 NIST SP 800-63B — Digital Identity Guidelines

Level	Application
AAL1	Username/password login
AAL2	MFA step-up for high-value transfers
AAL3	Facial liveness at onboarding (phishing-resistant enrollment)

7.3 OAuth & Identity Standards

Standard	Application
RFC 6749 — OAuth 2.0	Authorization framework via Auth0
RFC 7636 — PKCE	Public client security for SPA and mobile
RFC 9700 — OAuth 2.0 Security BCP	Authorization Code + PKCE (not Implicit)
OpenID Connect	ID tokens for identity, access tokens for API authorization

7.4 Data Retention Policy

Data Type	Retention	Storage	Immutable
Audit logs (client actions)	2 years minimum	S3 Object Lock (Compliance)	Yes
Transaction records	Per SB requirements (≥5 years recommended)	S3 + Core system	Yes (audit copy)
Biometric templates	Life of account + 1 year	DynamoDB (encrypted)	No (deletable on request)
Application logs	90 days hot, 1 year archive	CloudWatch → S3	No
Cache data	TTL-based (minutes to hours)	ElastiCache Redis	No

7.5 Compliance Architecture Diagram

COMPLIANCE CONTROLS			
LOPDP Privacy	SB Banking	OWASP App Sec	ISO 27001 ISMS
Auth0 (MFA) WAF Encryption Audit Pipeline DR Immutable S3 GuardDuty IAM EventBridge Monitoring			

7.6 Regulatory Notification Compliance

The assignment requires **minimum 2 notification systems** for movement alerts:

Channel	System	Regulatory basis
Email	Amazon SES	SB electronic banking — customer awareness of account activity
SMS	Amazon SNS / Pinpoint	Real-time alert for time-sensitive movements; higher reach in Ecuador where mobile penetration exceeds email usage

Both channels are triggered asynchronously by the **Notification Orchestrator** upon `transfer.completed` and `movement.detected` events, ensuring the transfer API response is not delayed by notification delivery.

7.7 Related ADRs

- [ADR-001: Auth0 and OAuth 2.0](#)
- [ADR-009: Immutable Audit](#)
- [ADR-005: MFA Step-Up](#)

8 Cost Estimate

Monthly order-of-magnitude cost breakdown for the proposed architecture at **peak scale** (tens of millions of transactions/day, thousands of active users). Estimates use **us-east-1** on-demand pricing (June 2025 reference); actual costs vary with reserved capacity, Savings Plans, and usage patterns.

Note: Budget is not a constraint for this design. Estimates reflect a **production-grade, HA, multi-region** deployment. Reserved Instances / Savings Plans could reduce compute costs by 30–50%.

8.1 Assumptions

Parameter	Value
Active users	~5,000 concurrent peak

Parameter	Value
Daily transactions	10–30 million (peak days)
API requests	~100 million/day (reads + writes + auth)
Audit events	~30 million/day
Notifications	~30 million/day (email + SMS combined)
Regions	us-east-1 (primary) + us-west-2 (DR warm standby)
Compute	ECS Fargate (0.5 vCPU / 1 GB per task, 3–20 tasks per service)

8.2 Monthly Cost Breakdown

AWS Service	Configuration	Est. Monthly (USD)	Notes
ECS Fargate	8 services × avg 10 tasks × 0.5 vCPU / 1 GB, 24/7	\$4,500 – \$7,000	Primary region; auto-scales to 20 tasks at peak
ECS Fargate (DR)	Warm standby: 8 tasks total in us-west-2	\$800 – \$1,200	Minimum viable standby
API Gateway	3 billion requests/month, REST API	\$10,500	\$3.50 per million requests
Application Load Balancer	2 ALBs (public + internal)	\$400 – \$600	LCU-based pricing
CloudFront	500 GB egress, 50M requests	\$200 – \$400	SPA static assets + API edge
AWS WAF	2 Web ACLs, OWASP rules, 3B requests	\$1,500 – \$2,500	\$1/million requests + rule fees
ElastiCache Redis	cache.r6g.xlarge × 3 shards, 2 replicas each	\$3,500 – \$4,500	Cluster mode, multi-AZ
DynamoDB	On-demand, ~500M reads + 100M writes/month	\$2,000 – \$3,500	Global tables add ~2× replication cost
Amazon EventBridge	1 billion events/month	\$1,000	\$1.00 per million events
Amazon SQS	500M messages/month	\$200 – \$400	Standard queues + DLQ
AWS Lambda	Audit pipeline: 30M invocations, 256 MB, 500 ms	\$150 – \$300	Pay per invocation
S3 Object Lock	5 TB audit storage, 30M PUTs/month	\$500 – \$800	Compliance retention
S3 CRR (DR)	Cross-region replication of audit bucket	\$300 – \$500	us-east-1 → us-west-2
Amazon SES	15M emails/month	\$1,500	\$0.10 per 1,000 emails
Amazon SNS (SMS)	15M SMS/month (Ecuador international rates)	\$4,500 – \$7,500	~\$0.30–0.50 per SMS to Ecuador
Amazon Rekognition	50K face liveness checks/month (onboarding)	\$200 – \$400	\$0.01 per liveness check
Secrets Manager	20 secrets	\$10	\$0.40/secret/month
CloudWatch	Logs (500 GB), metrics, alarms, dashboards	\$1,500 – \$2,500	
X-Ray	1 billion traces/month (sampled at 5%)	\$250 – \$500	
GuardDuty + Security Hub	Standard tier	\$300 – \$600	
Route 53	Hosted zone + health checks	\$50 – \$100	DR failover
VPC	NAT Gateway × 3 AZs, VPC endpoints	\$1,200 – \$1,800	\$0.045/hr per NAT + data processing
Auth0	Enterprise plan (external, not AWS)	\$1,500 – \$3,000	~1,000+ MAU enterprise tier

8.3 Total Estimate

Scenario	Monthly (USD)	Annual (USD)
Steady state (avg load)	\$25,000 – \$35,000	\$300K – \$420K
Peak month (campaign, payroll day)	\$35,000 – \$50,000	—
With Savings Plans (1-year)	\$18,000 – \$28,000	\$216K – \$336K

8.4 Cost by Category

Compute (ECS)		22%	\$6,300 – \$8,200
API & Networking		20%	\$12,400 – \$14,400
Cache & Database		15%	\$5,500 – \$8,000

6/19/26, 9:00 PM		Structurizr - BP Internet Banking	
Notifications (SMS)		15%	\$6,000 – \$9,000
Messaging & Events		10%	\$1,350 – \$1,700
Security & Monitoring		8%	\$2,550 – \$4,200
Storage & Audit		5%	\$800 – \$1,300
Auth0 (external)		5%	\$1,500 – \$3,000

8.5 Cost Optimization Opportunities

Strategy	Potential Savings	Trade-off
Compute Savings Plans (1-year)	30–40% on Fargate	Commitment
DynamoDB provisioned capacity	20–30% at predictable load	Capacity planning
SNS SMS → local SMS aggregator	40–60% on SMS	Additional vendor integration
X-Ray sampling at 1%	80% on tracing	Less trace coverage
S3 Intelligent-Tiering for audit	10–20% on storage	Slightly higher retrieval latency
Reserved ElastiCache nodes	30–40%	1-year commitment

8.6 Cost vs. Non-Functional Requirements

NFR	Cost Driver	Justification
99.95% SLA	Multi-AZ, warm DR, ALB	Cannot reduce without SLA impact
Low latency	ElastiCache, CloudFront	Cache cost << customer churn cost
Immutable audit	S3 Object Lock (Compliance)	Regulatory requirement; not negotiable
Dual notifications	SES + SNS SMS	Regulatory requirement
Security	WAF, GuardDuty, Security Hub	Cost of breach >> prevention cost

8.7 Scaling Cost Model

Daily Transactions Est. Monthly AWS Cost

1 million	\$8,000 – \$12,000
10 million	\$25,000 – \$35,000
30 million (peak)	\$35,000 – \$50,000
50 million	\$50,000 – \$70,000

Costs scale **sub-linearly** due to caching (higher hit rate at scale) and DynamoDB on-demand auto-scaling, but **SMS notifications scale linearly** with transaction volume.

8.8 Disclaimer

These are order-of-magnitude estimates for architecture evaluation purposes. Production costs require:

1. Detailed workload profiling (read/write ratio, payload sizes)
2. AWS Pricing Calculator with exact configurations
3. Proof-of-concept load testing to validate auto-scaling behavior
4. Negotiated Enterprise Discount Program (EDP) with AWS for volume commitments

Complete index of all 15 ADRs (ADR-001 → ADR-015).

9 Architecture Decision Records

9.1 ADR-001 — Auth0 with Authorization Code + PKCE

9.1.1 Status

Accepted

9.1.2 Context

BP requires OAuth 2.0 authentication for both the React SPA and React Native mobile app. The company has Auth0 as a configurable identity product. We must recommend the best OAuth flow per current security standards and support MFA, facial onboarding, and step-up authentication.

9.1.3 Decision

Use **Auth0** as the identity provider with the **Authorization Code + PKCE** flow for both SPA and mobile clients. Implement custom security policies via **Auth0 Actions**. Use **refresh token rotation** for session continuity.

Token validation at runtime: cryptographic JWT validation (signature, expiry, issuer, audience) is performed once at **API Gateway** (Auth0 JWT authorizer). The **BFF** performs scope and claim checks only (**transfer:write, acr**) — it does not re-validate the token. Details: ADR-003 (API Gateway + BFF).

9.1.4 Justification

OAuth 2.0 Security Best Current Practice (RFC 9700)

RFC 9700 explicitly deprecates the Implicit flow for SPAs and mandates PKCE for public clients. Authorization Code + PKCE prevents authorization code interception attacks by requiring a cryptographically random code verifier that never traverses the redirect channel. This is the only OAuth flow recommended for browser-based and native public clients in 2024+.

NIST SP 800-63B — Authenticator Assurance Levels

Auth0 supports graduated authentication assurance: AAL1 (password), AAL2 (MFA), and step-up challenges. This maps directly to banking requirements where routine reads need lower assurance than high-value transfers. Centralizing assurance levels in Auth0 avoids building custom MFA infrastructure.

9.1.5 Alternatives Considered

Alternative	Rejected Because
Implicit Flow	Deprecated by RFC 9700; access token exposed in URL fragment
Resource Owner Password Credentials	Credentials pass through client; no MFA hook; violates OAuth 2.0 BCP
Client Credentials	Machine-to-machine only; not for end-user banking
Self-built OAuth server (Keycloak)	Auth0 already licensed; building custom increases attack surface and maintenance
AWS Cognito	Would require migration from existing Auth0 investment; fewer banking-specific features
Azure AD B2C	Cloud preference is AWS; cross-cloud identity adds latency and complexity

9.1.6 Consequences

- **Positive:** Industry-standard security, MFA/Actions extensibility, reduced custom auth code
- **Negative:** Auth0 vendor dependency; JWKS validation at API Gateway only (~10ms with caching)
- **Follow-up:** Configure Auth0 Actions for device binding, risk scoring, and audit event publishing

9.2 ADR-002 — AWS as Cloud Provider

9.2.1 Status

Accepted

9.2.2 Context

BP requires cloud infrastructure with high availability, low latency for Ecuador users, multi-region DR, and budget for best-tier architecture. The assignment allows Azure or AWS.

9.2.3 Decision

Deploy on **Amazon Web Services (AWS)** with primary region **us-east-1** (multi-AZ) and DR region **us-west-2** (warm standby).

9.2.4 Justification

Architecture–platform fit (this design is AWS-native)

Every container in the C4 model maps to a first-class AWS managed service already chosen in other ADRs — not generic "cloud VMs." Azure offers equivalents, but adopting Azure would require remapping the whole design:

This architecture	AWS (chosen)	Azure equivalent
Immutable audit	S3 Object Lock	Blob immutability policies
Event fan-out	EventBridge → SQS	Event Grid → Service Bus
Facial onboarding	Rekognition	Azure AI Face
Notifications	SES + SNS	Communication Services
Compute	ECS Fargate	Container Apps / AKS
Idempotency / devices	DynamoDB	Cosmos DB

Staying on AWS avoids a full platform swap with no assignment requirement to compare both clouds in production.

Single regulated-data boundary (LODPD)

The platform processes **PII (Personally Identifiable Information)** — e.g. profile and onboarding data — **biometric data** (facial enrollment), and **immutable audit evidence**. Keeping compute, storage, messaging, biometrics, and audit in **one cloud IAM/VPC boundary** simplifies:

- Cross-border data transfer documentation under Ecuador's LODPD
- Unified CloudTrail + encryption + access policies
- No biometric images crossing to a second cloud for analysis

A split-cloud design (e.g. Azure compute + AWS Rekognition) adds compliance surface without benefit.

9.2.5 Alternatives Considered

Alternative	Rejected Because
Microsoft Azure	Capable alternative; would require remapping all containers above and swapping Rekognition → Azure Face, S3 Object Lock → Blob immutability, EventBridge → Event Grid, etc. No stated BP mandate or existing Azure investment to justify that rework
Google Cloud Platform	Smaller fintech reference base in LATAM; same full remapping cost as Azure
On-premises	Cannot meet multi-region DR, auto-scaling at peak, or elastic cost model
Hybrid cloud	Adds complexity; legacy Core/Detail are already REST-accessible
sa-east-1 (São Paulo) as primary	Evaluated; us-east-1 chosen for broader service availability (6 AZs) and DR pairing with us-west-2

9.2.6 Consequences

- **Positive:** Design, ADRs, cost estimate, and C4 model are internally consistent on one platform
- **Negative:** Vendor lock-in (mitigated by containerized workloads on ECS); LODPD cross-border processing agreement required for US regions
- **Follow-up:** Document data processing agreement for US-region storage under LODPD cross-border provisions

9.3 ADR-003 — API Gateway + BFF Integration Layer

9.3.1 Status

Accepted

9.3.2 Context

The assignment requires an API Gateway integration layer. Client applications (SPA, mobile) need a single entry point to backend services. Legacy Core and Detail systems must not be exposed directly to clients.

9.3.3 Decision

Implement **Amazon API Gateway** as the edge integration point with an **Auth0 JWT authorizer** (signature, expiry, issuer, audience validation). Route authenticated traffic to a **Banking BFF (Backend for Frontend)** that performs **scope and claim authorization** only — it does not re-validate the JWT cryptographically.

9.3.4 JWT Validation Split

Layer	Responsibility
API Gateway	Policy Enforcement Point (NIST SP 800-207): JWT authorizer validates token via Auth0 JWKS; rejects invalid/expired tokens before they reach backend
BFF Auth Middleware	Reads validated claims (sub , scope , acr); enforces authorization (e.g. transfer:write scope required for transfers); triggers step-up MFA via Risk Service when acr is insufficient

This avoids duplicate JWKS lookups on every request (~10ms each) while satisfying the assignment's API Gateway integration layer and OAuth 2.0 security requirements.

9.3.5 Justification

Backend for Frontend pattern (Sam Newman — "Building Microservices")

A BFF per client channel (or shared BFF for closely related channels) prevents domain services from being polluted with presentation logic. The BFF aggregates Profile + Detail data in one API call, reducing client round-trips from 3+ to 1. This directly improves mobile UX on high-latency networks (Ecuador → us-east-1).

API Gateway as cross-cutting concern enforcer (Gateway Offloading pattern — Enterprise Integration Patterns)

Centralizing JWT validation, rate limiting, throttling, and request logging at the gateway follows the Gateway Offloading pattern. This ensures security policies are applied consistently regardless of which backend service handles the request, adhering to the **Policy Enforcement Point** concept from zero-trust architecture (NIST SP 800-207).

9.3.6 Alternatives Considered

Alternative	Rejected Because
Direct client → microservices	Exposes internal service topology; no centralized throttling or auth
API Gateway → services (no BFF)	Clients make multiple calls; chatty on mobile; couples clients to service decomposition
GraphQL federation only	Adds complexity; REST + BFF aggregation sufficient for current scope
Kong / Apigee (self-managed)	Additional operational burden; API Gateway is managed and integrates with WAF/IAM
BFF re-validates JWT via JWKS	Redundant with gateway authorizer; doubles latency; no added security if gateway is the sole public entry point

9.3.7 Consequences

- **Positive:** Single security enforcement point, reduced client latency, clean separation of concerns
- **Negative:** BFF can become a bottleneck if not kept thin; requires discipline to avoid business logic creep
- **Follow-up:** BFF must only contain aggregation, auth, and routing — no domain rules

9.4 ADR-004 — Anti-Corruption Layer for Legacy REST Integration

9.4.1 Status

Accepted

9.4.2 Context

BP's Core Banking Platform and Client Detail System are legacy REST APIs with their own data models, error formats, and versioning. Domain microservices must consume this data without coupling to legacy internals.

9.4.3 Decision

Implement **Anti-Corruption Layer (ACL)** adapters inside each microservice that integrates with legacy systems (Core Banking Adapter, SWIFT Hub Adapter, Detail System Adapter). Wrap with **Circuit Breaker** and **Retry** patterns.

9.4.4 Justification

Anti-Corruption Layer (Eric Evans — Domain-Driven Design)

The ACL translates legacy API contracts into clean domain models understood by BP's microservices. Without it, legacy field names, error codes, and pagination quirks leak into every service, creating **model corruption** that makes refactoring impossible. The ACL is the bounded context boundary between BP's digital platform and legacy core systems.

Circuit Breaker pattern (Michael Nygard — "Release It!")

Legacy systems have unpredictable availability. The Circuit Breaker prevents thread exhaustion and cascading failures when Core returns timeouts or 5xx errors. After N failures, the circuit opens and returns fallback responses (cached data for reads, fail-fast for writes), preserving overall system availability per the **bulkhead** resilience principle.

9.4.5 Alternatives Considered

Alternative	Rejected Because
Direct REST calls from BFF	Couples integration layer to legacy; no fault isolation
Enterprise Service Bus (ESB)	Heavyweight, single point of failure; overkill for 2 legacy systems
Database replication from Core	Violates Core system ownership; stale data; regulatory issues
GraphQL wrapper over legacy	Still requires ACL underneath; adds layer without benefit
Shared integration library only	No runtime fault isolation; circuit breaker must be per-service

9.4.6 Consequences

- **Positive:** Legacy changes isolated to adapter; circuit breaker protects system stability
- **Negative:** Adapter code must be maintained when legacy APIs change; integration tests needed
- **Follow-up:** Contract tests against legacy API stubs; monitor circuit breaker state in CloudWatch

9.5 ADR-005 — MFA Step-Up for High-Risk Operations

9.5.1 Status

Accepted

9.5.2 Context

Banking transfers, especially high-value and interbank via SWIFT, require stronger authentication than routine balance checks. The assignment requires MFA for high-risk operations.

9.5.3 Decision

Implement a dedicated **Risk & MFA Gate Service** that evaluates risk scores before transfer execution. When the score exceeds configurable thresholds, trigger **Auth0 step-up authentication** requiring MFA (TOTP via authenticator app) before the transfer proceeds.

9.5.4 Justification

NIST SP 800-63B — Step-Up Authentication (Section 5.2)

NIST defines that sessions authenticated at AAL1 must re-authenticate to AAL2 before performing high-risk actions. Step-up authentication satisfies this by requiring a stronger authenticator mid-session without forcing full re-login. This is the industry-standard approach for banking transaction signing.

Risk-Based Authentication (OWASP ASVS V2.2)

OWASP Application Security Verification Standard requires re-authentication for sensitive operations based on contextual risk factors (amount, new payee, device trust, velocity). Separating risk evaluation into a dedicated service follows the **Single Responsibility Principle** (Robert Martin), allowing fraud rules to evolve independently of transfer logic.

9.5.5 Alternatives Considered

Alternative	Rejected Because
MFA on every login only	Does not protect against session hijacking post-login
MFA on every transfer	Poor UX for small routine transfers; increases abandonment
SMS OTP only	Lower assurance (SIM swap attacks); TOTP preferred per NIST
Transaction signing with hardware token	Physical token distribution cost prohibitive for retail banking
Risk rules embedded in Transfer Service	Violates SRP; fraud team cannot update rules without transfer service deployment
No MFA (password only)	Fails regulatory and security standards for financial institutions

9.5.6 Consequences

- **Positive:** Graduated security proportional to risk; regulatory compliance
- **Negative:** Additional latency (~2-5s) for MFA challenge on high-risk transfers
- **Follow-up:** Configure thresholds with fraud team; A/B test abandonment rates

9.6 ADR-006 — React Native for Mobile Banking

9.6.1 Status

Accepted

9.6.2 Context

The assignment requires a multiplatform mobile framework with two options evaluated and justified. BP also has a React SPA for web. Security-critical biometric and cryptographic operations are required.

9.6.3 Decision

Use **React Native** with **Kotlin native modules** for Android security operations (BiometricPrompt, Keystore, root detection). Compare against **Flutter** as the second evaluated option.

9.6.4 Justification

Conway's Law (Melvin Conway, 1968)

Organizations design systems that mirror their communication structure. With a React SPA team already in place, React Native allows the same team to share TypeScript types, API clients, testing practices, and component patterns. This reduces the **cognitive load** of maintaining two unrelated technology stacks (React + Dart/Flutter) and decreases defect rates from translation errors between teams.

Native Module security boundary (Defense in Depth)

React Native's architecture allows security-critical operations (biometric authentication, hardware keystore access, certificate pinning) to execute in **Kotlin native modules** that never cross the JavaScript bridge with sensitive key material. This implements **defense in depth** (NIST SP 800-53): the JS layer handles UI and business logic while the native layer handles cryptographic operations in hardware-backed secure storage.

9.6.5 Alternatives Considered

Alternative	Rejected Because
Flutter	No skill overlap with React SPA; platform channels less ergonomic for Kotlin-first Android security strategy
Native Swift + Kotlin (fully native)	2× development cost; violates multiplatform framework requirement
Ionic / Capacitor	WebView-based; inferior biometric integration and performance
Xamarin / .NET MAUI	No team expertise; smaller LATAM community
PWA	Cannot support facial enrollment camera access, biometric APIs, or app store security review

9.6.6 Consequences

- **Positive:** Code sharing with SPA, proven fintech track record (Nubank), native security modules
- **Negative:** Bridge performance for animation-heavy screens; requires Kotlin expertise for security modules
- **Follow-up:** See [Mobile Framework Comparison](#)

9.7 ADR-007 — Multi-Region DR (us-east-1 + us-west-2)

9.7.1 Status

Accepted

9.7.2 Context

The architecture must guarantee 99.95%+ availability, disaster recovery, and fault tolerance. User accepted US regions. Budget is not a constraint — best-tier architecture required.

9.7.3 Decision

Deploy primary workloads in **us-east-1** (multi-AZ) with **us-west-2** as DR region in **warm standby** configuration. Use Route 53 health-check failover, DynamoDB Global Tables, S3 Cross-Region Replication, and pre-provisioned ECS task definitions in DR.

9.7.4 Justification

ISO 22301 — Business Continuity Management

ISO 22301 requires that organizations identify critical business functions, define recovery strategies, and **test** recovery procedures. A warm standby DR region with quarterly failover drills satisfies the "tested recovery" requirement. Warm standby balances RTO (<30 min) against cost — superior to cold standby (hours RTO) and comparable to active-active at lower complexity.

AWS Well-Architected — Reliability Pillar (Disaster Recovery strategies)

AWS defines four DR strategies: backup/restore, pilot light, warm standby, and active-active. For 99.95% SLA with RTO <30 min and RPO <5 min, **warm standby** is the recommended strategy. It maintains minimum running capacity in DR (1 task per service) that scales to production on failover, avoiding the data synchronization complexity of active-active while meeting the SLA.

9.7.5 Alternatives Considered

Alternative	Rejected Because
Single region, multi-AZ only	Cannot survive regional outage (us-east-1 hurricane, fiber cut); fails 99.95% at regional scale
Active-active multi-region	Data consistency complexity for financial transactions; SWIFT hub connection duplication; 2× operational cost without proportional benefit
Cold standby (pilot light)	RTO >1 hour; fails 30-minute RTO target
On-prem DR site	Cannot auto-scale; capital expense; slower recovery
sa-east-1 primary	User accepted us-east-1; São Paulo adds latency without DR benefit over us-west-2

9.7.6 Consequences

- **Positive:** Regional failure survivability; meets 99.95% SLA and regulatory BC requirements
- **Negative:** ~\$800–1,200/month warm standby cost; DR drill operational overhead
- **Follow-up:** Quarterly failover drills; automate Route 53 failover with runbooks

9.8 ADR-008 — Facial Recognition for Mobile Onboarding

9.8.1 Status

Accepted

9.8.2 Context

Mobile onboarding requires facial recognition as part of the authorization and authentication flow. After onboarding, users may login with face, fingerprint, or username/password.

9.8.3 Decision

Use **Amazon Rekognition Face Liveness** for onboarding enrollment and optional face login. Store face templates encrypted in DynamoDB. Integrate via a dedicated **Onboarding & Identity Service** with Kotlin-native camera modules for capture quality.

9.8.4 Justification

Presentation Attack Detection — ISO/IEC 30107-3 (Biometric PAD)

ISO 30107-3 defines requirements for detecting presentation attacks (photos, videos, masks). Amazon Rekognition Face Liveness implements PAD-compliant liveness detection, preventing spoofing during enrollment. Without liveness, a photograph of the customer could bypass facial authentication, violating banking KYC requirements.

Separation of biometric processing (Privacy by Design — Ann Cavoukian)

Processing biometric data in a dedicated Onboarding Service, separate from transfer and movement services, implements **Privacy by Design** Principle 4 (full functionality without unnecessary data exposure). Biometric templates never reach the Transfer Service or BFF, minimizing the blast radius of a potential data breach and simplifying LOPDP compliance (data minimization).

9.8.5 Alternatives Considered

Alternative	Rejected Because
On-device only (no cloud)	Lower accuracy; no centralized liveness; harder to audit
FaceTec / Onfido (third-party)	Additional vendor cost; AWS Rekognition integrates natively with AWS stack
Azure Face API	Cloud preference is AWS; cross-cloud latency
Store raw face images	LOPDP violation (data minimization); storage cost; breach risk
Facial login only (no password fallback)	Accessibility and device compatibility concerns; user must have choice
Skip facial, password only	Fails assignment requirement for facial onboarding

9.8.6 Consequences

- **Positive:** PAD-compliant liveness, AWS-native integration, isolated biometric processing
- **Negative:** Rekognition cost per check; network dependency for face login; privacy impact assessment required under LOPDP
- **Follow-up:** LOPDP biometric data impact assessment; encrypt templates at rest; never log raw images

9.9 ADR-009 — Immutable Audit with S3 Object Lock

9.9.1 Status

Accepted

9.9.2 Context

The assignment requires an audit database recording all client actions. Banking regulations (SB Ecuador) require tamper-evident transaction records. Retention is approximately 2 years.

9.9.3 Decision

Implement an **event-driven audit pipeline** (EventBridge → Lambda → S3 Object Lock) with **Compliance mode** retention. Use DynamoDB as a queryable index for audit lookups. All client actions publish domain events consumed by the audit pipeline.

9.9.4 Justification

WORM storage for regulatory compliance (SEC 17a-4 / FINRA principles)

Financial regulators worldwide require that audit records cannot be altered or deleted before retention expiry. Amazon S3 Object Lock in **Compliance mode** provides Write Once Read Many (WORM) semantics — not even AWS root account can delete records before retention period. This satisfies the **integrity** and **non-repudiation** requirements of banking audit standards.

Event Sourcing for audit trail completeness (Martin Fowler — Event Sourcing)

By capturing every domain event (login, movement viewed, transfer initiated, transfer completed, MFA challenge) as an immutable event, the audit trail is a natural byproduct of system operation rather than a bolt-on logging concern. Event Sourcing ensures **completeness** (no action bypasses the event bus) and **temporal accuracy** (events are timestamped at occurrence, not at log write time).

9.9.5 Alternatives Considered

Alternative	Rejected Because
Amazon QLDB	Valid alternative; higher cost at 30M events/day; S3 Object Lock is more cost-effective at scale
RDS PostgreSQL audit table	Mutable by DBA; no WORM guarantee; harder to prove non-tampering to auditors
CloudWatch Logs only	Not WORM; 1-year max retention in standard; not suitable for regulatory evidence
Application-level logging only	Bypassable if service compromised; no centralized guarantee
Blockchain audit	Over-engineered; operational complexity unjustified for this scale
No retention policy	Fails regulatory requirements

9.9.6 Consequences

- **Positive:** Tamper-evident, cost-effective at scale, event-driven (no performance impact on transfer path)
- **Negative:** S3 Compliance mode retention cannot be shortened even by admins; query latency higher than database (mitigated by DynamoDB index)
- **Follow-up:** Define canonical audit schema; implement audit query API for compliance team

9.10 ADR-010 — ADR-010-cache-aside-redis

9.10.1 Status

Accepted

9.10.2 Context

The assignment requires a persistence mechanism for frequent clients. At millions of transactions/day with thousands of users, profile and movement reads must be low-latency. Legacy Core REST calls add 50–200ms per request.

9.10.3 Decision

Implement **Cache-Aside** pattern using **Amazon ElastiCache for Redis** (cluster mode, multi-AZ). Cache customer profiles, recent movements, and product summaries with configurable TTLs. Profile and Movements services manage their own cache read/write logic.

9.10.4 Justification

Cache-Aside pattern (Martin Fowler — Patterns of Enterprise Application Architecture)

Cache-Aside gives the application explicit control over what is cached and when, unlike write-through or write-behind caches that couple cache and database updates. For banking reads where data freshness matters (movements change on transfer), the application can set **TTL per data type** (profile: 15 min, recent movements: 2 min) and invalidate on write events. This implements **eventual consistency** with bounded staleness — acceptable for read paths per BASE theorem (Basically Available, Soft state, Eventually consistent).

Pareto principle (80/20 rule) applied to access patterns

In retail banking, ~20% of customers generate ~80% of login frequency (frequent clients checking balances daily). Caching this hot subset in Redis (sub-10ms reads) reduces Core REST load by an estimated 60–80%, directly supporting the tens-of-millions peak transaction throughput without scaling legacy systems. This is the **economic justification** for cache investment: ElastiCache cost (~\$4K/month) is far less than Core system scaling or customer churn from slow UX.

9.10.5 Alternatives Considered

Alternative	Rejected Because
Read-through cache	Cache manages DB reads; less control over TTL per data type; harder invalidation
Write-through cache	Every write hits cache + Core; adds latency to transfer path
CDN caching for API	Cannot cache authenticated, per-user financial data at edge
DynamoDB DAX	Only accelerates DynamoDB, not legacy Core REST calls
Application in-memory cache	Not shared across ECS tasks; cache miss on every task scale-out
Materialized views in Core	Requires legacy system changes; outside BP's control
CQRS with dedicated read DB	Higher complexity; justified only if cache hit rate insufficient

9.10.6 Consequences

- **Positive:** 60–80% reduction in Core REST calls; sub-10ms reads for frequent clients; horizontal scaling via Redis cluster
- **Negative:** Cache invalidation complexity; stale data risk if TTL too long (mitigated by event-driven invalidation on transfers)
- **Follow-up:** Monitor cache hit rate (target >85%); implement cache invalidation on transfer.completed events

9.11 ADR-011 — ADR-011-saga-transfers

9.11.1 Status

Accepted

9.11.2 Context

Transfers may involve multiple steps: validate accounts, debit source, submit to SWIFT hub (interbank), credit destination, notify customer, audit. These span BP microservices and external systems without distributed transaction support.

9.11.3 Decision

Implement the **Saga pattern** (orchestration-based) in the Transfer Service using a **Saga Orchestrator** component. Each step has a corresponding **compensation action** for rollback on failure.

9.11.4 Justification

Saga pattern for distributed transactions (Chris Richardson — Microservices Patterns)

The Saga pattern manages data consistency across services without two-phase commit (2PC), which is impractical over REST APIs and external SWIFT hubs. An orchestrator sequences steps (debit → SWIFT → credit) and triggers compensating transactions (credit back source) on failure. This provides **eventual consistency** with explicit failure handling — the only viable approach for heterogeneous distributed systems (CAP theorem: partition tolerance requires sacrificing atomic consistency).

Idempotency for safe retries (Designing Data-Intensive Applications — Martin Kleppmann)

At millions of transfers/day, network timeouts will cause clients to retry. The Saga combined with **DynamoDB idempotency keys** ensures that a retried transfer request does not execute twice. Kleppmann demonstrates that idempotency keys are essential for exactly-once **effect** in at-least-once delivery systems — critical for financial operations where duplicate debits are unacceptable.

9.11.5 Alternatives Considered

Alternative	Rejected Because
Two-Phase Commit (2PC)	Not supported by REST APIs or SWIFT hub; blocks resources; single point of failure
Choreography-based Saga	Harder to trace and debug; transfer flow is sequential and well-defined (orchestration fits better)
Single monolithic transfer	Cannot scale independently; couples SWIFT integration with Core integration
Fire-and-forget to Core	No compensation on failure; money could be debited without SWIFT submission
Eventual consistency without compensation	Unacceptable for financial transactions; must reverse partial completions

9.11.6 Consequences

- **Positive:** Reliable multi-step transfers with rollback; idempotent retries; auditable saga state
- **Negative:** Saga state management complexity; compensation may fail (requires manual intervention runbook)
- **Follow-up:** Implement saga state machine in DynamoDB; dashboard for stuck sagas; manual compensation runbook

9.12 ADR-012 — ADR-012-event-driven-async

9.12.1 Status

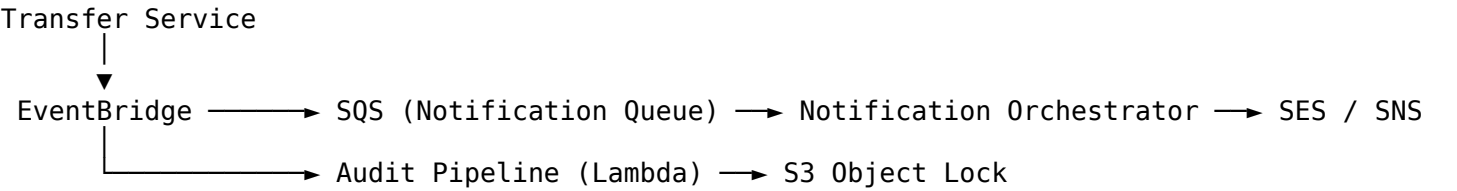
Accepted

9.12.2 Context

Regulatory requirements mandate customer notification on movements (email + SMS). All client actions must be audited. Both must not block the transfer API response path.

9.12.3 Decision

Use **Amazon EventBridge** as the domain event bus. The **Transfer Service** publishes `transfer.completed` and `transfer.failed` events after saga completion. EventBridge **fans out** to two independent consumers:



- **Notification path:** EventBridge rule targets **SQS (notification queue)**. The Notification Orchestrator polls SQS, renders templates, and dispatches to **SES** (email) and **SNS** (SMS). Failed deliveries retry with exponential backoff; poison messages go to a **DLQ**.
- **Audit path:** EventBridge invokes the **Audit Pipeline** (Lambda) for all domain events. Records are normalized and written to **S3 Object Lock** (immutable) with a DynamoDB index.

The Transfer Service returns success to the customer **before** notifications are delivered or audit records are persisted (eventual consistency).

9.12.4 Justification

Temporal Decoupling (Enterprise Integration Patterns — Gregor Hohpe)

The notification requirement means the transfer service must not wait for email/SMS delivery before responding. EventBridge provides **temporal decoupling**: the Transfer Service publishes an event and immediately returns. SQS adds **buffering and retry** between

EventBridge and the Notification Orchestrator, so a slow or temporarily unavailable SES/SNS endpoint does not lose events. This satisfies the **responsiveness** quality attribute while maintaining **reliability** via SQS retries and DLQ.

Open/Closed Principle (Bertrand Meyer — SOLID)

New consumers (fraud analytics, CRM, regulatory reporting) can subscribe to existing EventBridge events without modifying the Transfer Service. The fan-out pattern keeps the system **open for extension** (new rules and targets) but **closed for modification** (transfer logic unchanged). This supports the assignment requirement for future component addition.

9.12.5 Alternatives Considered

Alternative	Rejected Because
Synchronous notification in Transfer Service	Adds 500ms–2s latency (SES/SMS); transfer timeout risk; violates responsiveness
EventBridge → Notification Service directly (no SQS)	No buffering; consumer downtime loses events; harder retry/DLQ management
Direct SQS from Transfer Service	Point-to-point; each new consumer requires Transfer Service changes
Kafka (MSK)	Higher operational cost and complexity; EventBridge sufficient at this scale
Database polling for notifications	Anti-pattern; adds DB load; delayed delivery
Webhook callbacks from Transfer Service	Tight coupling; no retry/DLQ; fails if consumer down

9.12.6 Consequences

- **Positive:** Non-blocking transfers, extensible event consumers, SQS buffering with built-in retry/DLQ, parallel audit and notification paths
- **Negative:** Eventual notification delivery (seconds delay acceptable per regulations); event ordering not guaranteed (mitigated by idempotency)
- **Follow-up:** Define event schema registry; monitor notification delivery SLA separately from transfer SLA in CloudWatch

9.13 ADR-013 — Kotlin for Backend Microservices

9.13.1 Status

Accepted

9.13.2 Context

The assignment allows Kotlin for Android and backend. BP has six domain microservices (Profile, Movements, Transfer, Onboarding, Risk, Notification) plus a Node.js/TypeScript BFF. Teams with Java and Spring Boot experience can adopt Kotlin quickly — same JVM, same Spring Boot stack, less boilerplate than Java.

Transfer remains the most security-critical and concurrency-intensive service (SWIFT interbank transfers with saga orchestration). Standardizing all domain services on Kotlin simplifies operations and aligns backend code with Kotlin Android native modules.

9.13.3 Decision

Implement **all six domain microservices** in **Kotlin with Spring Boot**:

Service	Notes
Customer Profile Service	Legacy Core/Detail adapters, cache-aware reads
Movements Service	Paginated history, circuit breakers
Transfer Service	Saga orchestration; coroutines for non-blocking step execution
Onboarding & Identity Service	Facial enrollment, Auth0 provisioning; shared Kotlin with mobile native modules
Risk & MFA Gate Service	Fraud rules, step-up triggers
Notification Orchestrator	Async email/SMS fan-out

Keep **Banking BFF** on **Node.js / TypeScript** (channel aggregation, shared with React SPA).

Use a **shared Kotlin library** for ACL adapters, idempotency helpers, and Resilience4j patterns across services.

9.13.4 Justification

JVM language uniformity without losing Spring Boot (Pragmatic Standardization)

Kotlin compiles to JVM bytecode and runs on the same Spring Boot, ECS Fargate, and observability stack already chosen for BP. Developers with Java/Spring Boot skills ramp on Kotlin quickly — no new runtime, no second deployment model. One backend language reduces context switching, shared libraries, and CI/CD templates compared to a Java + Kotlin split.

Null safety for financial and PII paths (Tony Hoare's billion-dollar mistake)

Kotlin's nullable vs non-nullable types are enforced at compile time. This reduces null-related defects in account numbers, amounts, client identifiers, and biometric references across Profile, Movements, Transfer, and Onboarding — not only in the transfer path.

Coroutines where concurrency matters most (CSP model — Tony Hoare)

Transfer Service saga steps (debit → SWIFT → credit) use Kotlin **coroutines** for structured concurrency with fewer threads than Java `CompletableFuture` chains. Other services remain largely request/response; they benefit from Kotlin expressiveness without requiring coroutines everywhere.

Alignment with mobile security modules (Defense in Depth)

Onboarding and Android native modules (BiometricPrompt, Keystore) already use Kotlin. A Kotlin backend for Onboarding and shared security utilities avoids translating types and validation rules between Java services and Kotlin mobile code.

9.13.5 Alternatives Considered

Alternative	Rejected Because
Java for all domain services	Valid; more boilerplate; split stack with Kotlin mobile modules; no compile-time null safety
Kotlin for Transfer + Onboarding only	Previous approach; operational and hiring overhead of two JVM languages for similar Spring Boot services
Node.js for domain services	BFF already uses Node for channel layer; saga, legacy ACL, and regulated data paths fit JVM/Spring better
Go / Rust	No team expertise; smaller Spring/integration ecosystem for legacy REST adapters
Mixed Java + Kotlin	Works technically; rejected in favor of one domain language and shared Kotlin libraries

9.13.6 Consequences

- **Positive:** Single JVM language for domain tier; null-safe financial logic; coroutines for Transfer saga; Kotlin shared with Android modules; simpler CI/CD and library reuse
- **Negative:** Team must be comfortable with Kotlin idioms (coroutines, extension functions); slightly smaller hiring pool than pure Java (offset by Java → Kotlin ramp-up)
- **Follow-up:** Shared Kotlin module for ACL, idempotency, and Resilience4j; coroutine-based saga template in Transfer Service

9.14 ADR-014 — ADR-014-microservice-decomposition

9.14.1 Status

Accepted

9.14.2 Context

The assignment requires 3 initial services (basic data, movements, transfers) with permission to add more. The architecture must be decoupled, cohesive, and extensible.

9.14.3 Decision

Decompose into **6 domain microservices** plus integration layer:

Service	Responsibility
Customer Profile Service	Basic + detail client data
Movements Service	Transaction history
Transfer Service	Own-account and SWIFT transfers
Onboarding & Identity Service	Facial enrollment, device binding
Risk & MFA Gate Service	Risk scoring, step-up triggers
Notification Orchestrator	Email + SMS delivery

9.14.4 Justification

Single Responsibility Principle (Robert C. Martin — SOLID)

Each service has one reason to change: Profile changes when Core integration changes; Transfer changes when SWIFT rules change; Risk changes when fraud rules change. Combining these into fewer services would mean a fraud rule update requires redeploying the transfer engine — unacceptable operational risk in a 99.95% SLA environment.

Independent Deployability (Sam Newman — "Building Microservices")

The core benefit of microservices is **independent deployability**: each service can be built, tested, deployed, and scaled independently. Movements Service (read-heavy, cacheable) scales differently from Transfer Service (write-heavy, saga-bound). At tens of millions of peak transactions, independent scaling prevents over-provisioning read services to match write demand.

9.14.5 Alternatives Considered

Alternative	Rejected Because
3 services only (as minimum)	Onboarding, Risk, and Notifications are required by assignment but don't fit cleanly into the 3 core services
Monolith	Cannot scale reads vs writes independently; deployment coupling
1 service per legacy system	Couples BP logic to legacy boundaries rather than domain boundaries
Nano-services (20+)	Operational overhead; network latency; unjustified at this scale
Modular monolith	Valid for smaller scale; insufficient for multi-region DR and independent scaling at peak

9.14.6 Service Addition Justification

Added Service	Why Beyond the Required 3
Onboarding & Identity	Assignment requires facial onboarding; isolates biometric and PII (Personally Identifiable Information) processing
Risk & MFA Gate	Assignment requires MFA step-up; fraud rules change frequently
Notification Orchestrator	Assignment requires 2 notification channels; async decoupling from transfers

9.14.7 Consequences

- **Positive:** Independent scaling, deployment, and failure isolation; clear team ownership boundaries
- **Negative:** 6 services to operate; distributed tracing essential; integration testing complexity
- **Follow-up:** Define service ownership matrix; implement contract testing between services

9.15 ADR-015 — ADR-015-cloudwatch-xray-observability

9.15.1 Status

Accepted

9.15.2 Context

9.15.3 Decision

Implement the **three observability pillars** with two AWS-native services:

Pillar	Tool	Scope
Metrics + Logs	Amazon CloudWatch	SLIs/SLOs, structured JSON application logs, dashboards, alarms (P1–P4)
Traces	AWS X-Ray	Distributed tracing from API Gateway → BFF → microservices → legacy adapters (5% sampling)

All ECS Fargate services, API Gateway, and the Audit Lambda emit logs and custom metrics to CloudWatch. X-Ray SDK instruments the request path across services; sampling at 5% balances cost (~\$250–500/month) with sufficient coverage for latency debugging.

GuardDuty and CloudTrail remain complementary security monitoring (see `regulatory.md` §5) — not substitutes for application observability.

9.15.4 Justification

AWS Well-Architected — Operational Excellence pillar

CloudWatch is the native integration point for ECS Fargate, API Gateway, Lambda, and EventBridge. Using managed observability avoids operating a separate stack (Prometheus/Grafana) and keeps alarms, auto-scaling triggers, and dashboards in one place. Structured JSON logs with `correlationId` enable end-to-end request tracing when combined with X-Ray trace IDs.

Distributed tracing necessity (Google SRE — "Monitoring Distributed Systems")

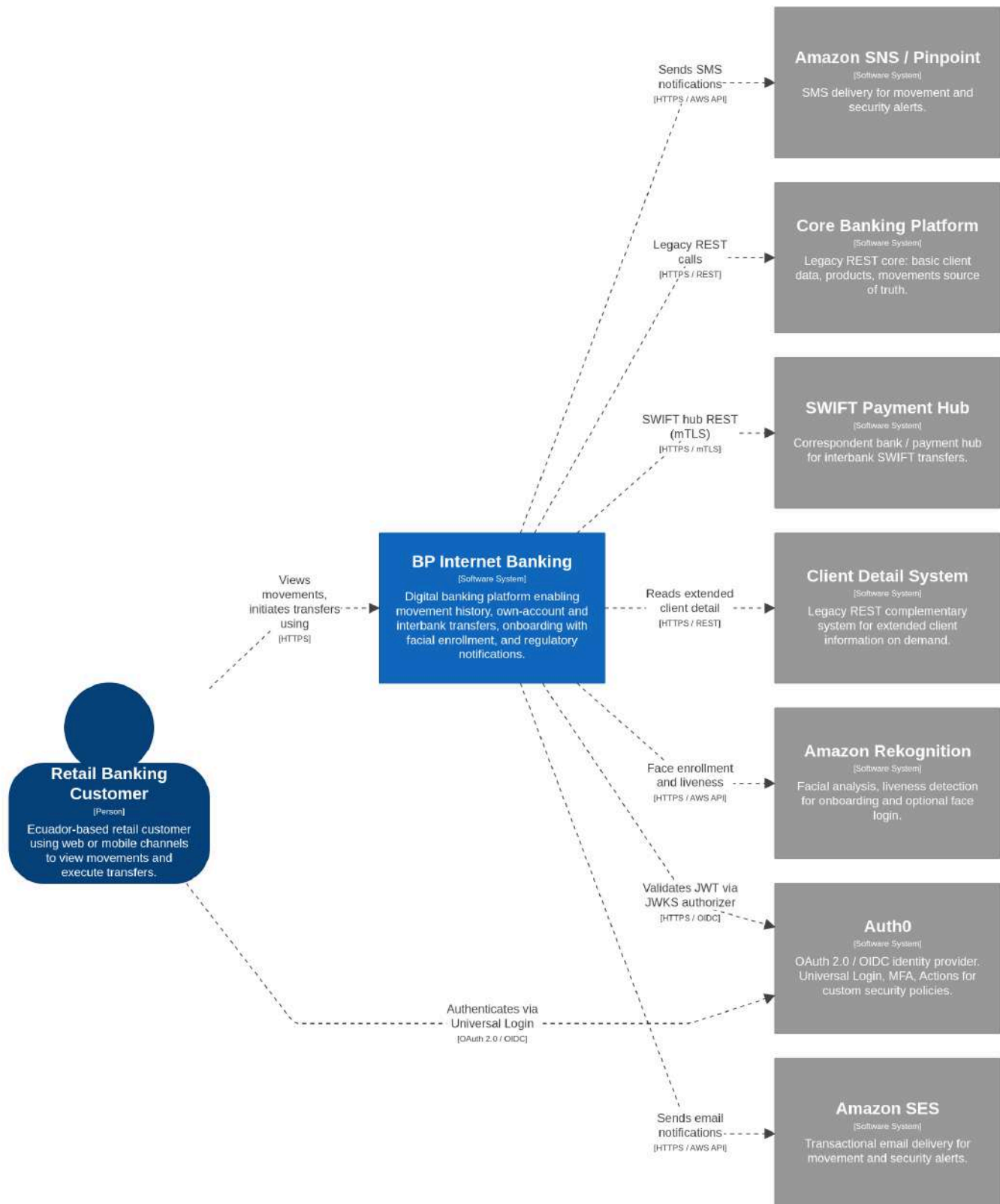
In a microservices architecture with BFF aggregation and legacy adapter calls, p99 latency issues are invisible in per-service logs alone. X-Ray reveals which hop (BFF → Transfer → Core adapter → SWIFT) contributes latency. Sampling at 5% is sufficient at tens-of-millions tx/day scale while controlling cost — a pragmatic trade-off for a budget-conscious deployment.

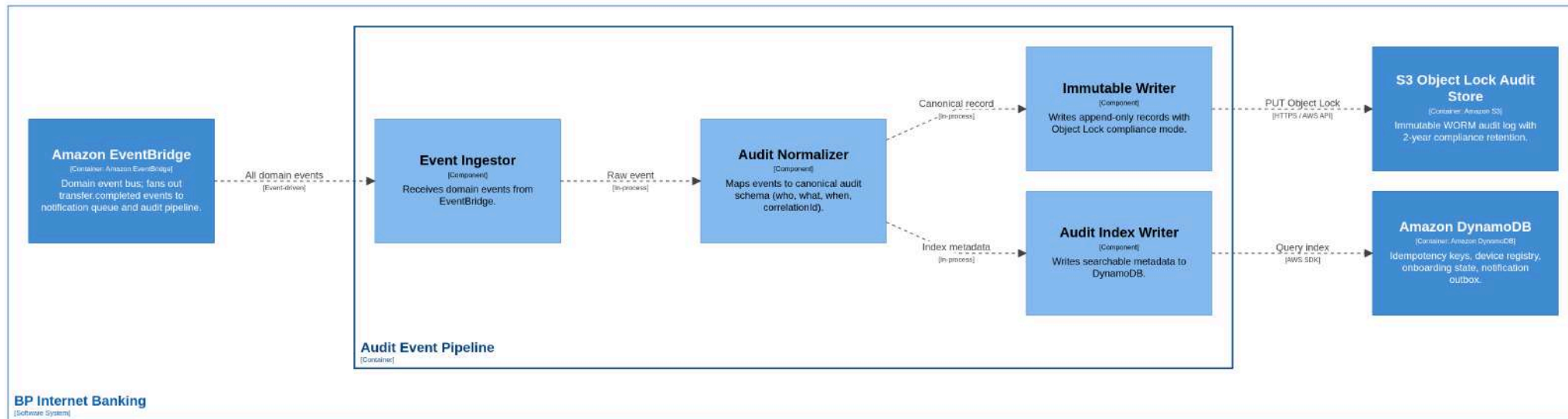
9.15.5 Alternatives Considered

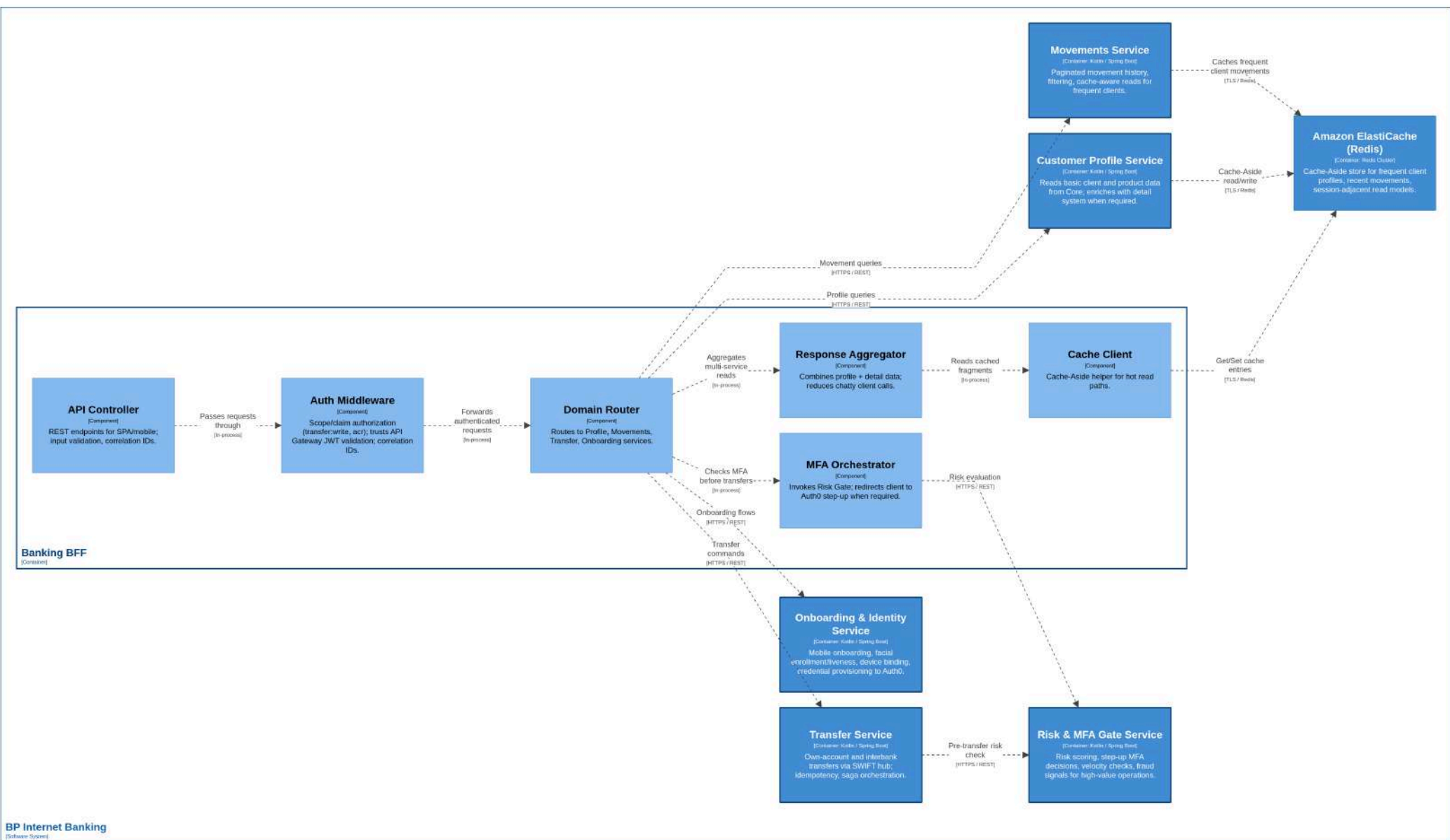
Alternative	Rejected Because
CloudWatch only (no tracing)	Cannot diagnose cross-service latency; insufficient for microservices debugging
Datadog / New Relic	Additional vendor cost (\$5K+/month); CloudWatch + X-Ray covers assignment scope
OpenTelemetry + self-hosted Jaeger	Operational burden; ECS sidecar complexity
X-Ray at 100% sampling	Cost scales linearly with traffic; 5% sufficient for banking SLO monitoring
Application logs to S3 only	No real-time alerting; delayed incident detection

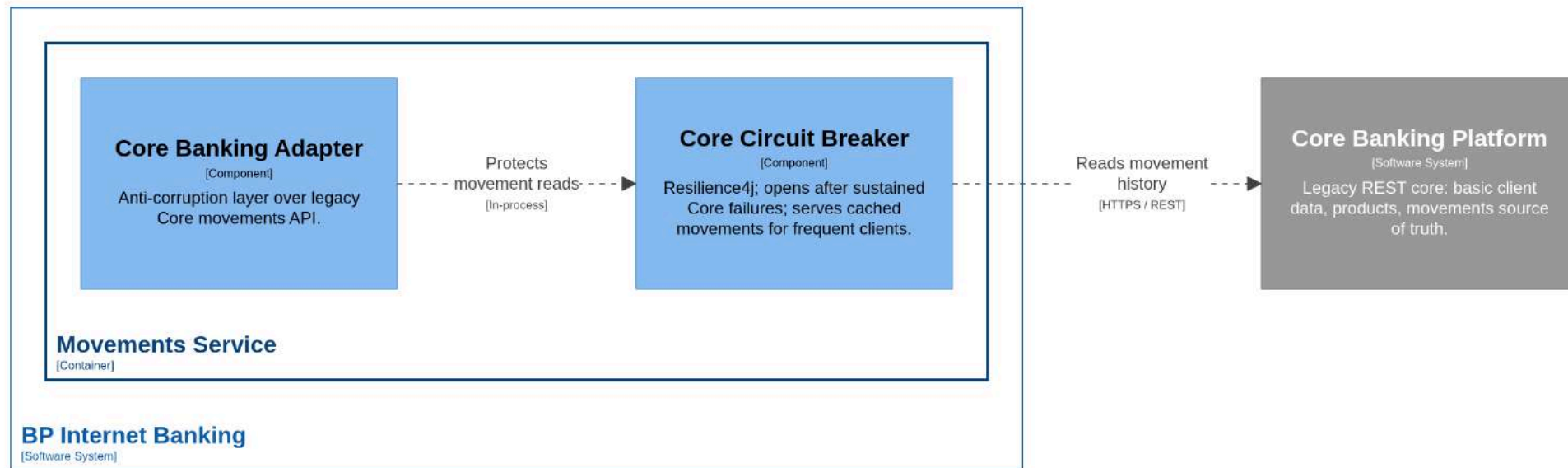
9.15.6 Consequences

- **Positive:** Unified AWS-native monitoring, distributed trace visibility, alarm-driven incident response
- **Negative:** X-Ray 5% sampling may miss rare tail events; CloudWatch log volume cost at scale (mitigated by retention tiers)
- **Follow-up:** Define SLO dashboards (Customer Experience, Transfer Health, Security); wire P1 alarms to PagerDuty





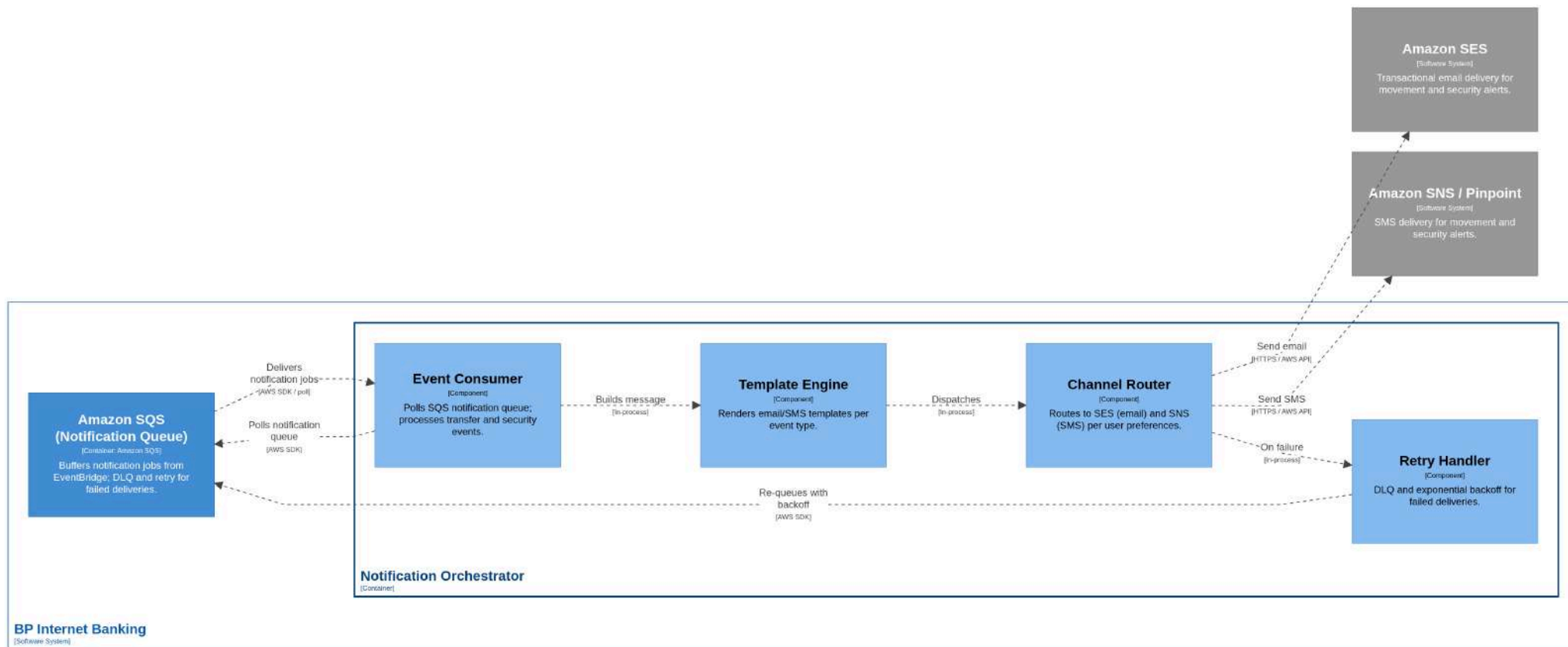




Component View: BP Internet Banking - Movements Service

Components — Movements Service (history queries, cache-aware reads)

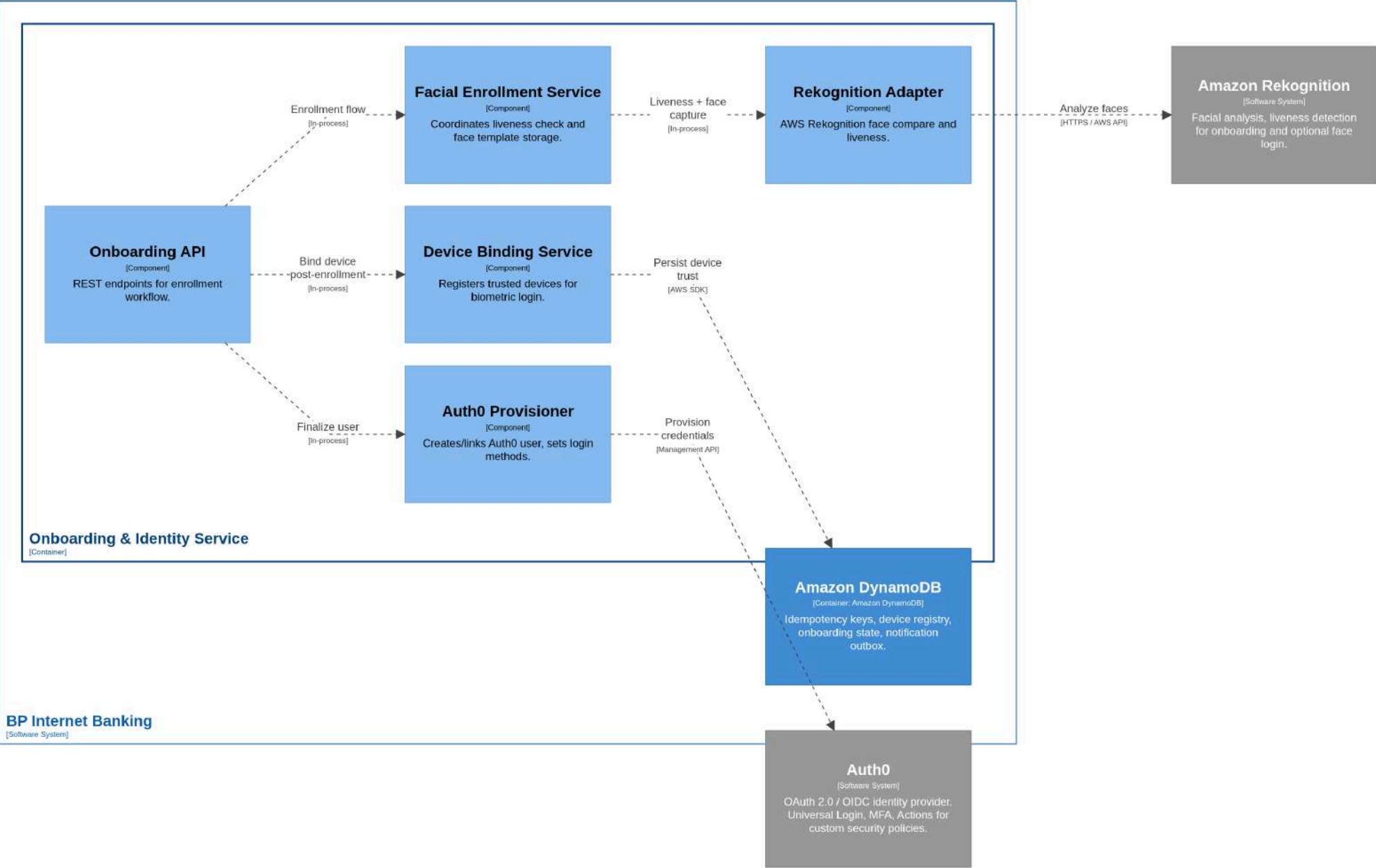
Friday, June 19, 2026 at 8:59 PM Ecuador Time



Component View: BP Internet Banking - Notification Orchestrator

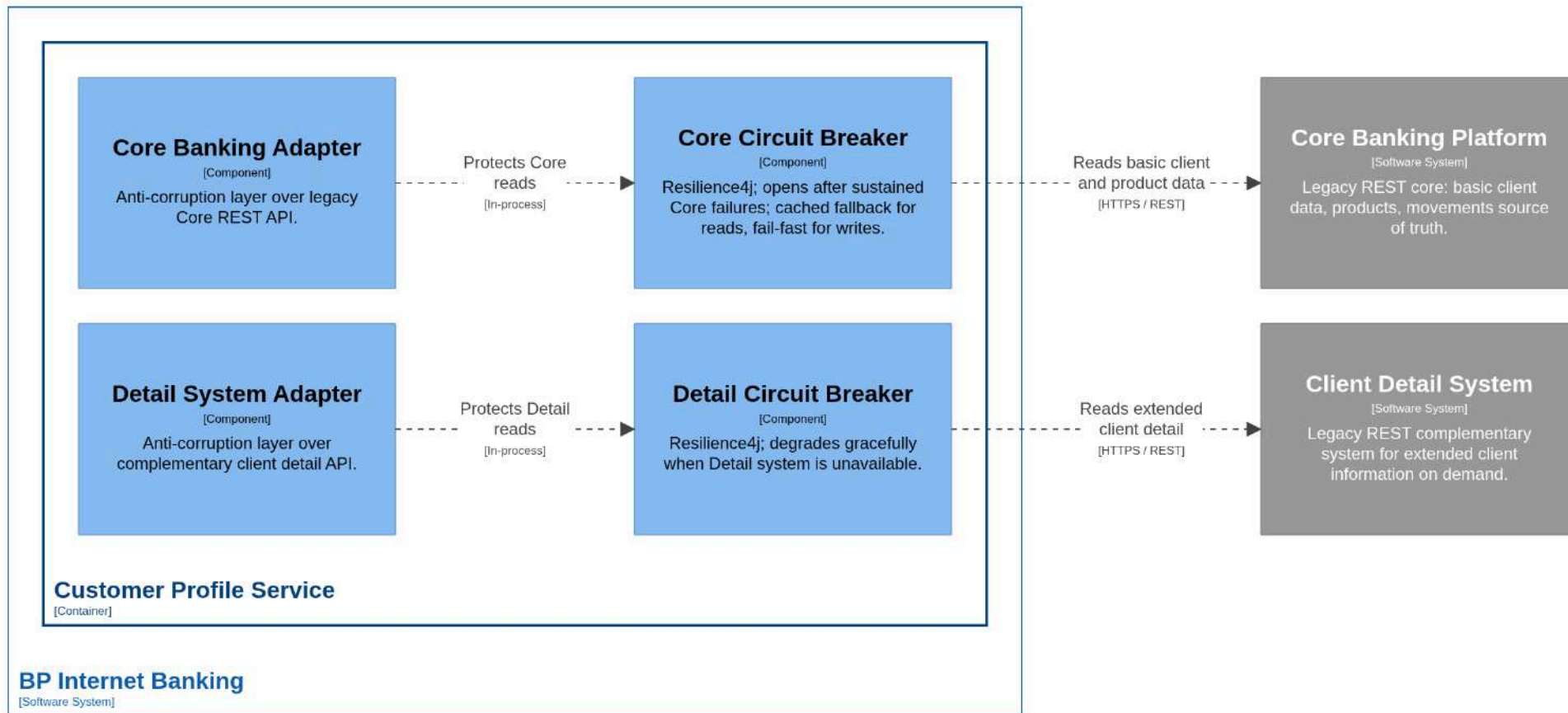
Components — Notification Orchestrator (email + SMS)

Friday, June 19, 2026 at 8:59 PM Ecuador Time



Component View: BP Internet Banking - Onboarding & Identity Service

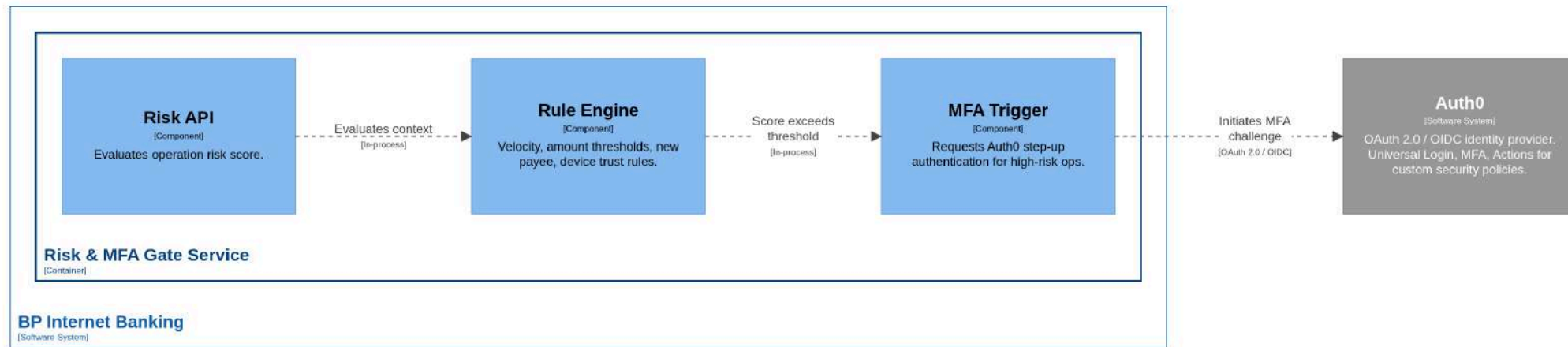
Components — Onboarding & Identity (facial enrollment, device binding)



Component View: BP Internet Banking - Customer Profile Service

Components — Customer Profile Service (Core + Detail integration)

Friday, June 19, 2026 at 8:10 PM Ecuador Time



Component View: BP Internet Banking - Risk & MFA Gate Service

Components — Risk & MFA Gate

Friday, June 19, 2026 at 8:59 PM Ecuador Time

